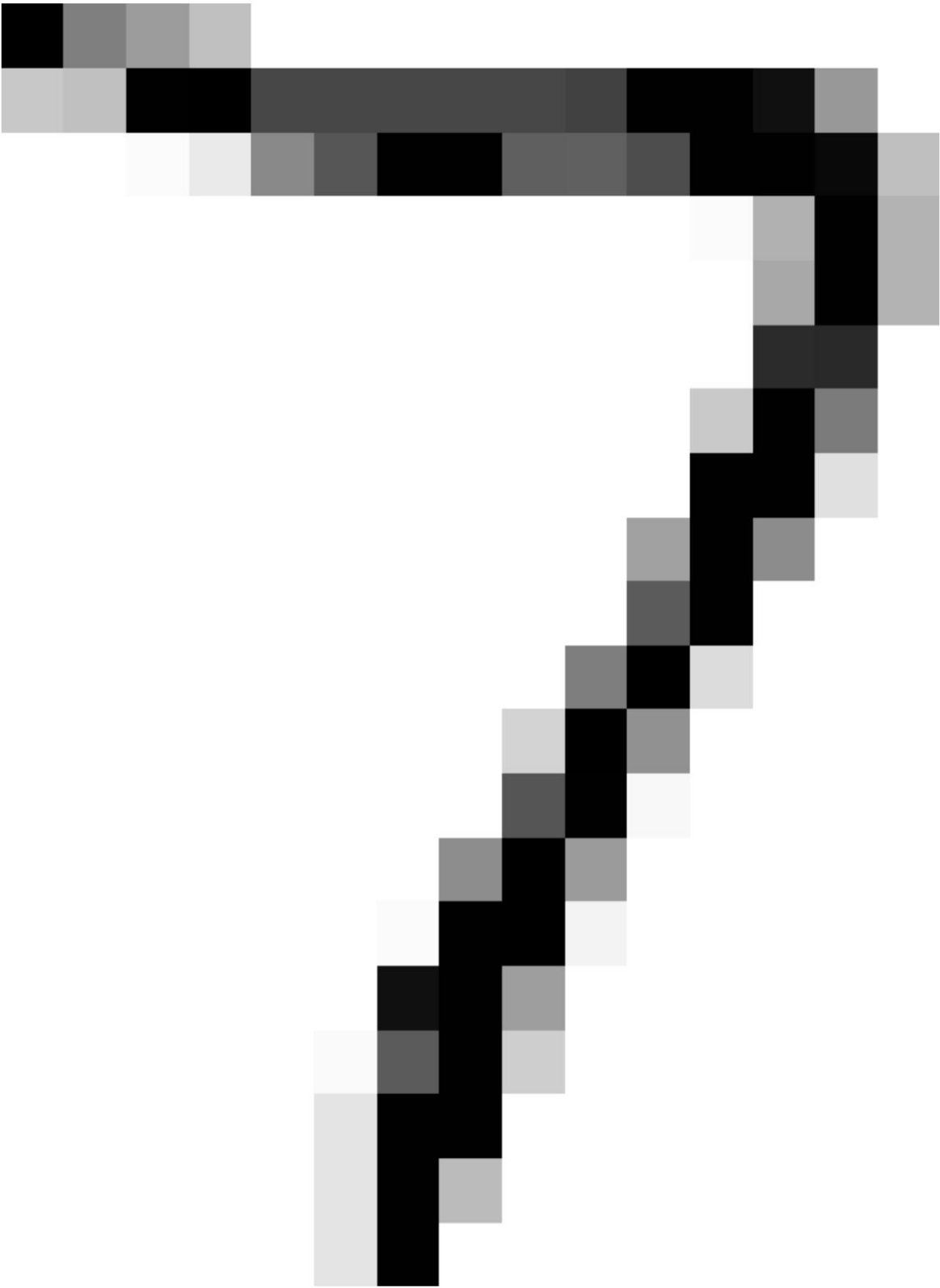


Bên trái là ảnh đầu vào bị nhiễu, và bên phải là ảnh mục tiêu đã được làm sạch. Bây giờ chúng ta hãy huấn luyện bộ phân loại và làm cho nó làm sạch ảnh này (Hình 3-13):

```
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train_mod, y_train_mod)
clean_digit = knn_clf.predict([X_test_mod[0]])
plot_digit(clean_digit)
plt.show()
```



Hình 3-13. Hình ảnh đã được làm sạch.

Có vẻ khá gần với mục tiêu rồi! Vậy là chúng ta đã kết thúc phần tìm hiểu về phân loại. Giờ đây bạn đã biết cách chọn các chỉ số tốt cho các tác vụ phân loại, chọn tỷ lệ chính xác/độ thu hồi phù hợp, so sánh các thuật toán phân loại, và nói chung là xây dựng các hệ thống phân loại tốt cho nhiều nhiệm vụ khác nhau.

Trong các chương tiếp theo, chúng ta sẽ xem xét cách thức hoạt động thực tế của tất cả các mô hình Học máy mà chúng ta đã sử dụng.

Bài tập

1. Hãy thử xây dựng một bộ phân loại cho tập dữ liệu MNIST đạt độ chính xác trên 97% trên tập dữ liệu thử nghiệm. Gợi ý: bộ phân loại `KNeighborsClassifier` hoạt động khá tốt cho nhiệm vụ này; bạn chỉ cần tìm các giá trị siêu tham số phù hợp (thử tìm kiếm tối ưu trên lưới cho các siêu tham số `weights` và `n_neighbors`).
2. Viết một hàm có thể dịch chuyển ảnh MNIST theo bất kỳ hướng nào (trái, phải, lên hoặc xuống) một pixel. Sau đó, với mỗi ảnh trong tập huấn luyện, tạo bốn bản sao đã dịch chuyển (một bản cho mỗi hướng) và thêm chúng vào tập huấn luyện. Cuối cùng, huấn luyện mô hình tốt nhất của bạn trên tập huấn luyện mở rộng này và đo độ chính xác của nó trên tập kiểm tra. Bạn sẽ thấy rằng mô hình của bạn hoạt động thậm chí còn tốt hơn! Kỹ thuật mở rộng tập huấn luyện một cách nhân tạo này được gọi là tăng cường dữ liệu hoặc mở rộng tập huấn luyện.
3. Giải quyết bộ dữ liệu Titanic. Kaggle là một nơi tuyệt vời để bắt đầu . Ngoài ra, bạn cũng có thể tải dữ liệu từ <https://homl.info/titanic.tgz> Và giải nén tệp tarball này giống như bạn đã làm với dữ liệu nhà ở trong **Chương 2**. Thao tác này sẽ tạo ra hai tệp CSV: `train.csv` và `test.csv` , bạn có thể tải chúng bằng `pandas.read_csv()`. Mục tiêu là huấn luyện một bộ phân loại có thể dự đoán cột "Sống sót" dựa trên các cột khác.
4. Xây dựng bộ phân loại thư rác (bài tập khó hơn):

- Tải xuống các ví dụ về thư rác và thư không phải thư rác từ bộ dữ liệu công khai của [Apache SpamAssassin](#).
- Giải nén các tập dữ liệu và làm quen với định dạng dữ liệu.
- Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra.
- Hãy viết một quy trình chuẩn bị dữ liệu để chuyển đổi mỗi email thành một vectơ đặc trưng. Quy trình chuẩn bị của bạn nên biến đổi một email thành một vectơ (thưa) cho biết sự hiện diện hoặc vắng mặt của mỗi từ có thể có. Ví dụ, nếu tất cả các email chỉ chứa bốn từ, "Hello", "how", "are", "you", thì email "Hello you Hello Hello you" sẽ được chuyển đổi thành vectơ $[1, 0, 0, 1]$ (có nghĩa là ["Hello" có mặt, "how" vắng mặt, "are" vắng mặt, "you" có mặt]), hoặc $[3, 0, 0, 2]$ nếu bạn muốn đếm số lần xuất hiện của mỗi từ.

Bạn có thể muốn thêm các siêu tham số vào quy trình chuẩn bị của mình để kiểm soát việc có nên loại bỏ tiêu đề email, chuyển đổi từng email thành chữ thường, xóa dấu câu, thay thế tất cả URL bằng "URL", thay thế tất cả số bằng "NUMBER" hay không, hoặc thậm chí thực hiện việc rút gọn từ (tức là cắt bỏ phần cuối của từ; có các thư viện Python có sẵn để làm điều này).

Cuối cùng, hãy thử nghiệm với một vài thuật toán phân loại khác nhau và xem liệu bạn có thể xây dựng một thuật toán phân loại thư rác tuyệt vời, với cả độ thu hồi cao và độ chính xác cao hay không.

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://homl.info/colab3>.

-
1. Theo mặc định, Scikit-Learn lưu trữ các tập dữ liệu đã tải xuống trong một thư mục có tên `scikit_learn_data` trong thư mục chính của bạn.
 2. Các tập dữ liệu được trả về bởi `fetch_openml()` không phải lúc nào cũng được xáo trộn hoặc chia nhỏ.
 3. Việc xáo trộn có thể là một ý tưởng tồi trong một số trường hợp—ví dụ, nếu bạn đang làm việc với dữ liệu chuỗi thời gian (như giá cổ phiếu hoặc điều kiện thời tiết). Chúng ta sẽ tìm hiểu điều này trong [Chương 15](#).
 4. Các bộ phân loại Scikit-Learn luôn có một trong hai phương thức: `decision_function()` hoặc `predict_proba()`. Phương thức `predict_proba()`, hoặc đôi khi cả hai.

5. Scikit-Learn cung cấp một vài tùy chọn tính trung bình khác và các chỉ số phân loại đa nhãn; xem phần tiếp theo. Xem tài liệu để biết thêm chi tiết.
6. Bạn có thể sử dụng hàm `shift()` từ `scipy.ndimage.interpolation` mô-đun. Ví dụ, `shift(image, [2, 1], cval=0)` dịch chuyển hình ảnh xuống hai pixel và sang phải một pixel.

Chương 4. Mô hình huấn luyện

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất-nội dung thô và chưa chỉnh sửa của tác giả khi họ viết-nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 4 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Cho đến nay, chúng ta đã xem các mô hình Học máy và thuật toán huấn luyện của chúng như những hộp đen. Nếu bạn đã làm một số bài tập trong các chương trước, bạn có thể ngạc nhiên về việc bạn có thể làm được bao nhiêu việc mà không cần biết gì về cơ chế hoạt động bên trong: bạn đã tối ưu hóa một hệ thống hồi quy, cải thiện bộ phân loại hình ảnh chữ số, và thậm chí xây dựng một bộ phân loại thư rác từ đầu, tất cả mà không cần biết chúng hoạt động như thế nào. Thật vậy, trong nhiều trường hợp, bạn thực sự không cần biết chi tiết triển khai.

Tuy nhiên, việc hiểu rõ cách mọi thứ hoạt động sẽ giúp bạn nhanh chóng xác định được mô hình phù hợp, thuật toán huấn luyện đúng đắn và bộ siêu tham số tốt cho nhiệm vụ của mình. Hiểu được những gì diễn ra bên trong cũng sẽ giúp bạn gỡ lỗi và phân tích lỗi hiệu quả hơn. Cuối cùng, hầu hết các chủ đề được thảo luận trong chương này sẽ rất cần thiết để hiểu, xây dựng và huấn luyện mạng nơ-ron (được thảo luận trong **Phần II** của cuốn sách này).

Trong chương này, chúng ta sẽ bắt đầu bằng việc xem xét mô hình Hồi quy tuyến tính, một trong những mô hình đơn giản nhất. Chúng ta sẽ thảo luận về hai cách rất khác nhau để huấn luyện mô hình này:

- Sử dụng phương trình "dạng đóng" để tính¹ toán trực tiếp các tham số mô hình sao cho phù hợp nhất với tập dữ liệu huấn luyện (tức là các tham số mô hình giúp giảm thiểu hàm chi phí trên tập dữ liệu huấn luyện).
- Sử dụng phương pháp tối ưu hóa lặp gọi là Gradient Descent (GD), phương pháp này dần dần điều chỉnh các tham số của mô hình để giảm thiểu hàm chi phí trên tập dữ liệu huấn luyện, cuối cùng hội tụ về cùng một tập hợp tham số như phương pháp đầu tiên. Chúng ta sẽ xem xét một vài biến thể của Gradient Descent mà chúng ta sẽ sử dụng đi sử dụng lại khi nghiên cứu mạng nơ-ron trong **Phần II**: Batch GD, Mini-batch GD và Stochastic GD.

Tiếp theo, chúng ta sẽ xem xét Hồi quy Đa thức, một mô hình phức tạp hơn có thể phù hợp với các tập dữ liệu phi tuyến tính. Vì mô hình này có nhiều tham số hơn Hồi quy Tuyến tính, nên nó dễ bị quá khớp dữ liệu huấn luyện hơn. Do đó, chúng ta sẽ xem xét cách phát hiện xem điều này có xảy ra hay không bằng cách sử dụng đường cong học tập, và sau đó chúng ta sẽ xem xét một số kỹ thuật điều chỉnh có thể giảm nguy cơ quá khớp tập dữ liệu huấn luyện.

Cuối cùng, chúng ta sẽ xem xét thêm hai mô hình thường được sử dụng cho các nhiệm vụ phân loại: Hồi quy Logistic và Hồi quy Softmax.

CẢNH BÁO

Chương này sẽ có khá nhiều phương trình toán học, sử dụng các khái niệm cơ bản về đại số tuyến tính và giải tích. Để hiểu các phương trình này, bạn cần biết vectơ và ma trận là gì; cách chuyển vị, nhân và nghịch đảo chúng; và đạo hàm riêng là gì. Nếu bạn chưa quen thuộc với các khái niệm này, vui lòng tham khảo các bài hướng dẫn giới thiệu về đại số tuyến tính và giải tích có sẵn dưới dạng sổ tay Jupyter trong tài [liệu bổ sung trực tuyến](#). Đối với những ai thực sự dị ứng với toán học, bạn vẫn nên đọc chương này và bỏ qua phần phương trình; hy vọng rằng phần văn bản sẽ đủ để giúp bạn hiểu hầu hết các khái niệm.

Hồi quy tuyến tính

Trong **Chương 1**, chúng ta đã xem xét một mô hình hồi quy đơn giản về sự hài lòng trong cuộc sống: sự hài lòng trong cuộc sống = $\theta_0 + \theta_1 \times$ GDP bình quân đầu người.

Mô hình này chỉ là một hàm tuyến tính của biến đầu vào GDP_per_capita. θ_0 và θ_1 là các tham số của mô hình. 1
các thông số.

Nói một cách tổng quát hơn, mô hình tuyến tính đưa ra dự đoán bằng cách đơn giản tính tổng có trọng số của các đặc trưng đầu vào. cộng thêm một hằng số được gọi là số hạng sai lệch (còn được gọi là số hạng chặn), như được thể hiện trong **Phương trình 4-1**.

Phương trình 4-1. Dự đoán mô hình hồi quy tuyến tính

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

Trong phương trình này:

- $\hat{y}$ là giá trị dự đoán.
- n là số lượng đặc điểm.
- x_i là giá trị th của thuộc tính i .
- θ_j là tham số mô hình thứ j , bao gồm số hạng độ lệch θ_0 và các trọng số đặc trưng $\theta_1, \dots, \theta_n$.

Điều này có thể được viết ngắn gọn hơn nhiều bằng cách sử dụng dạng vectơ, như thể hiện trong **Phương trình 4-2**.

Phương trình 4-2. Dự đoán mô hình hồi quy tuyến tính (dạng vectơ)

$$\hat{y} = h(x) = \theta^T x$$

Trong phương trình này:

- h là hàm giả thuyết, sử dụng các tham số mô hình θ .
- θ là vectơ tham số của mô hình, bao gồm số hạng độ lệch θ_0 và các trọng số đặc trưng $\theta_1, \dots, \theta_n$.
- x là vectơ đặc trưng của đối tượng, chứa các giá trị từ x_0 đến x_n , với x_0 luôn bằng 1.
- $\theta^T \cdot x$ là tích vô hướng của hai vectơ θ và x , bằng $\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$.

GHI CHÚ

Trong Học máy, vectơ thường được biểu diễn dưới dạng vectơ cột, là mảng 2 chiều với một cột duy nhất. Nếu θ và x là vectơ cột. Nếu θ là vectơ, thì dự đoán là $\hat{y} = \theta^T x$, trong đó θ^T là ma trận chuyển vị của θ (vectơ hàng thay vì vectơ cột) và x là ma trận. phép nhân của θ^T và x . Tất nhiên, đó vẫn là dự đoán tương tự, ngoại trừ việc giờ đây nó được biểu diễn dưới dạng ma trận một ô thay vì một ma trận nhiều ô. Giá trị vô hướng. Trong cuốn sách này, tôi sẽ sử dụng ký hiệu này để tránh việc chuyển đổi giữa tích vô hướng và phép nhân ma trận.

Được rồi, đó là mô hình Hồi quy tuyến tính – nhưng làm thế nào để huấn luyện nó? Hãy nhớ rằng, huấn luyện một mô hình có nghĩa là... thiết lập các tham số sao cho mô hình phù hợp nhất với tập dữ liệu huấn luyện. Để làm được điều này, trước tiên chúng ta cần một thước đo về Mức độ phù hợp (hoặc không phù hợp) của mô hình với dữ liệu huấn luyện. Trong **Chương 2**, chúng ta đã thấy rằng hiệu suất phổ biến nhất Chỉ số đánh giá của một mô hình hồi quy là Sai số bình phương trung bình gốc (RMSE) (**Phương trình 2-1**). Do đó, để huấn luyện một Trong mô hình hồi quy tuyến tính, chúng ta cần tìm giá trị của θ sao cho RMSE được tối thiểu hóa. Trên thực tế, cách đơn giản hơn là... giảm thiểu sai số bình phương trung bình (MSE) hơn là sai số bình phương trung bình gốc (RMSE), và nó dẫn đến cùng một kết quả (vì giá trị đó (Nó tối thiểu hóa một hàm số dương đồng thời cũng tối thiểu hóa căn bậc hai của nó).

CẢNH BÁO

Các thuật toán học máy thường tối ưu hóa một hàm mất mát khác trong quá trình huấn luyện so với thước đo hiệu năng được sử dụng để đánh giá mô hình cuối cùng. Điều này thường là do hàm đó dễ tối ưu hóa hơn và/hoặc vì nó có thêm các thành phần cần thiết chỉ trong quá trình huấn luyện (ví dụ: để điều chỉnh). Một thước đo hiệu năng tốt là càng gần với mục tiêu kinh doanh cuối cùng càng tốt. Một hàm mất mát huấn luyện tốt là hàm dễ tối ưu hóa và có mối tương quan mạnh mẽ với thước đo. Ví dụ, các bộ phân loại thường được huấn luyện bằng cách sử dụng một hàm chi phí như hàm mất mát logarit (như chúng ta sẽ thấy sau trong chương này) nhưng được đánh giá bằng độ chính xác/độ thu hồi. Hàm mất mát logarit dễ dàng được tối thiểu hóa, và việc làm như vậy thường sẽ cải thiện độ chính xác/độ thu hồi.

Sai số bình phương trung bình (MSE) của giả thuyết hồi quy tuyến tính h trên tập huấn luyện X được tính bằng **phương trình 4-3**.

Phương trình 4-3. Hàm chi phí MSE cho mô hình hồi quy tuyến tính

MSE(X, h) = 1 / m ∑ (x (i) - y (i)) ^ 2

Hầu hết các ký hiệu này đã được trình bày trong **Chương 2** (xem **"Ký hiệu"**). Sự khác biệt duy nhất là chúng ta viết h thay vì chỉ h để làm rõ rằng mô hình được tham số hóa bởi vectơ θ. Để đơn giản hóa ký hiệu, chúng ta sẽ chỉ viết MSE(θ) thay vì MSE(X, h).

θ

Phương trình chuẩn

Để tìm giá trị của θ sao cho MSE được tối thiểu hóa, tồn tại một nghiệm dạng đóng— nói cách khác, một phương trình toán học cho kết quả trực tiếp. Phương trình này được gọi là Phương trình Chuẩn (**Phương trình 4-4**).

Phương trình 4-4. Phương trình chuẩn

θ ^ = (X ^ T X) ^ - 1 X ^ T y

Trong phương trình này:

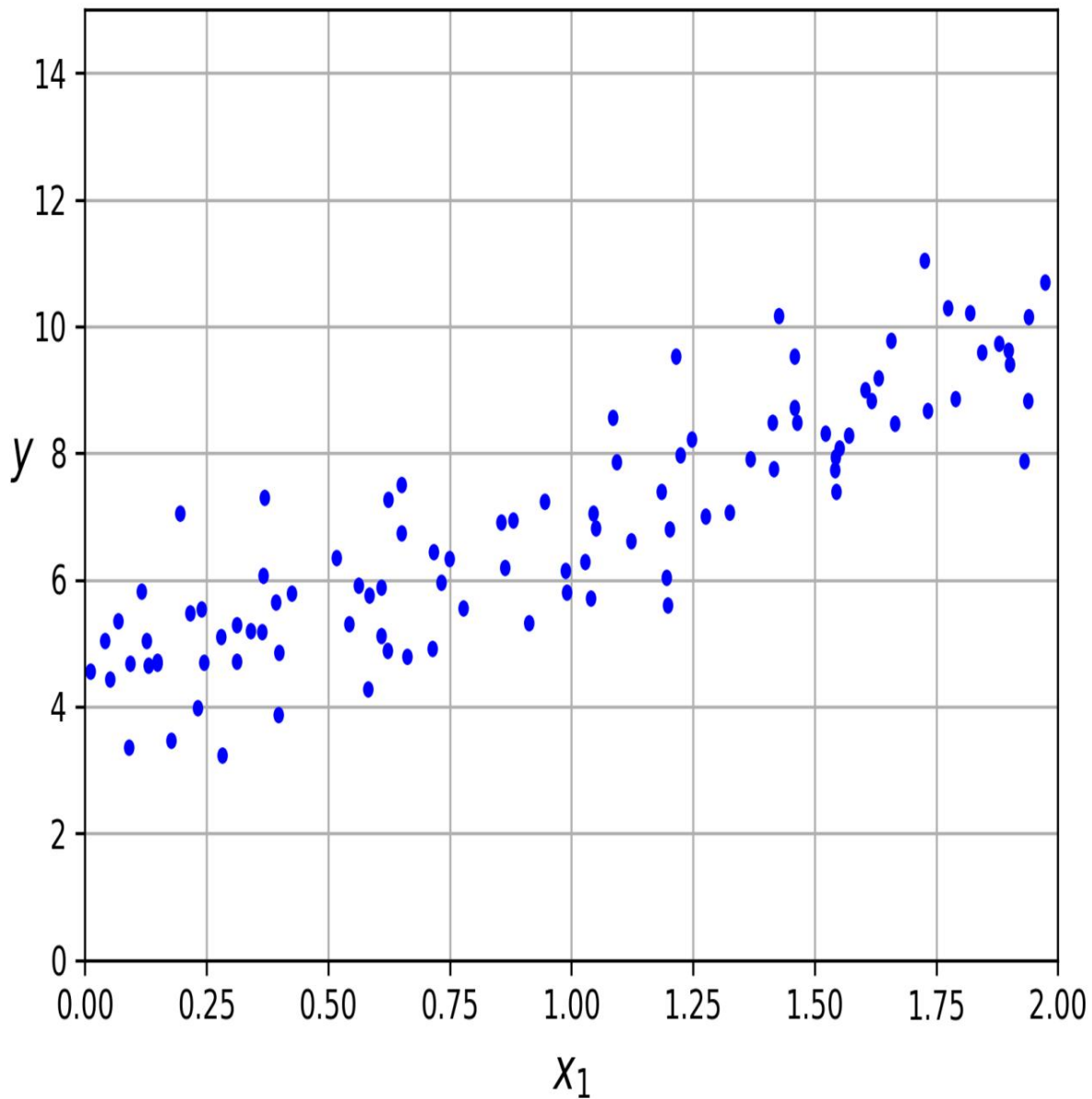
- θ ^ là giá trị của θ làm giảm thiểu hàm chi phí.
- y là vectơ các giá trị mục tiêu chứa y đến y . (1) (m)

Hãy tạo ra một số dữ liệu có dạng tuyến tính để kiểm tra phương trình này (**Hình 4-1**):

```
import numpy as np

np.random.seed(42) # để làm cho ví dụ mã này có thể tái tạo được m = 100 # số lần xuất hiện

# np.random.rand(m, 1) # vectơ cột X = 2 y = 4 + 3
# X + np.random.randn(m, 1) # vectơ cột
```

Hình 4-1. Tập dữ liệu tuyến tính được tạo ngẫu nhiên

Bây giờ chúng ta hãy $\hat{y}$ sử dụng phương trình chuẩn. Chúng ta sẽ sử dụng hàm `inv()` từ thư viện tuyến tính của NumPy. tính toán mô-đun đại số (`np.linalg`) để tính ma trận nghịch đảo và phương thức `dot()` để nhân ma trận:

```
from sklearn.preprocessing import add_dummy_feature

X_b = add_dummy_feature(X) # thêm x0 = 1 vào mỗi trường hợp
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
```

GHI CHÚ

Toán tử `@` thực hiện phép nhân ma trận. Nếu `A` và `B` là mảng NumPy, thì `A @ B` tương đương với `np.matmul(A, B)`. Nhiều thư viện khác như TensorFlow, PyTorch hoặc JAX cũng hỗ trợ toán tử `@`. Tuy nhiên, bạn không thể sử dụng `@` trên các mảng Python thuần túy (tức là danh sách các danh sách).

Hàm số mà chúng ta đã sử dụng để tạo ra dữ liệu là $y = 4 + 3x + \text{nhiều Gaussian}$. Hãy xem phương trình này tìm ra kết quả gì:

```
>>> theta_best
array([[4.21509616],
       [2.77011339]])
```

Chúng tôi đã hy vọng nhận được $\theta = 4$ và θ_3 thay vì $\theta = 4,215$ và $\theta = 2,770$. Kết quả cũng khá gần, nhưng nhiễu đã khiến việc khôi phục chính xác các tham số của hàm gốc trở nên bất khả thi. Tập dữ liệu càng nhỏ và càng nhiễu, việc này càng khó khăn.

Giờ đây chúng ta có thể đưa ra dự đoán bằng cách sử dụng $\hat{\cdot}$:

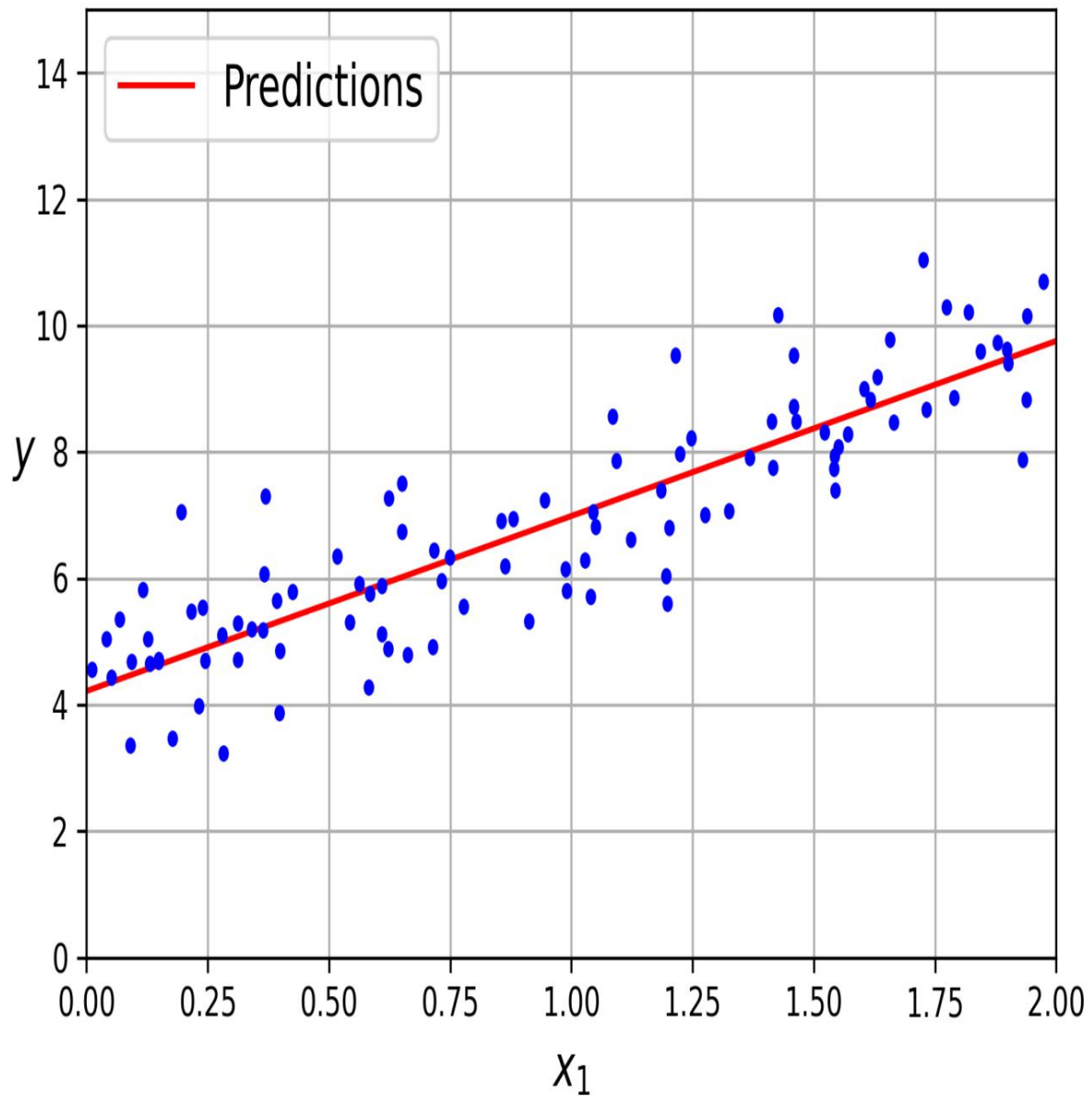
```
>>> X_new = np.array([[0], [2]])
>>> X_new_b = add_dummy_feature(X_new) # thêm x0 = 1 vào mỗi trường hợp >>> y_predict =
X_new_b @ theta_best >>> y_predict

array([[4.21509616],
       [9.75532293]])
```

Hãy vẽ đồ thị thể hiện các dự đoán của mô hình này (Hình 4-2):

```
import matplotlib.pyplot as plt

plt.plot(X_new, y_predict, "r-", label="Dự đoán") plt.plot(X, y,
"b.") [...] # Làm đẹp
hình: thêm nhãn, trục, lưới và chú thích plt.show()
```



Hình 4-2. Dự đoán của mô hình hồi quy tuyến tính

Thực hiện hồi quy tuyến tính bằng Scikit-Learn tương đối đơn giản:

```
>>> from sklearn.linear_model import LinearRegression >>> lin_reg =
LinearRegression() >>> lin_reg.fit(X, y) >>>
lin_reg.intercept_,
lin_reg.coef_ (array([4.21509616]), array([[2.77011339]]))
>>> lin_reg.predict(X_new) array([[4.21509616], [9.75532293]])
```

Lưu ý rằng Scikit-Learn tách hệ số thiên vị (intercept_) khỏi trọng số đặc trưng (coef_). Lớp LinearRegression dựa trên hàm `scipy.linalg.lstsq()` (tên gọi này viết tắt của “phương pháp bình phương nhỏ nhất”), mà bạn có thể gọi trực tiếp:

```
>>> theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6) >>>
theta_best_svd
array([[4.21509616],
       [2.77011339]])
```

Hàm này tính toán nghịch X^+ = X^+y , trong đó X^+ là nghịch đảo giả của X (cụ thể là ma trận Moore-Penrose) đảo. Bạn có thể sử dụng `np.linalg.pinv()` để tính toán nghịch đảo giả trực tiếp:

```
>>> np.linalg.pinv(X_b) @ y
array([[4.21509616],
       [2.77011339]])
```

Bản thân ma trận giả nghịch đảo được tính toán bằng cách sử dụng kỹ thuật phân tích ma trận chuẩn gọi là Phân tích Giá trị Đơn (SVD), có thể phân tích ma trận huấn luyện X thành phép nhân ma trận của ba ma trận U , Σ , V (xem `numpy.linalg.svd()`). Ma trận giả nghịch đảo được tính là $X^+ = V\Sigma^+U^T$. Để tính ma trận Σ^+ , thuật toán lấy Σ và đặt tất cả các giá trị nhỏ hơn một ngưỡng ϵ rất nhỏ bằng 0, sau đó nó thay thế tất cả các giá trị khác 0 bằng nghịch đảo của chúng, và cuối cùng nó chuyển vị ma trận kết quả. Cách tiếp cận này hiệu quả hơn so với việc tính toán Phương trình Chuẩn, cộng thêm nó xử lý tốt các trường hợp ngoại lệ: thực tế, Phương trình Chuẩn có thể không hoạt động nếu ma trận XX^T không khả nghịch (tức là ma trận đơn), chẳng hạn như nếu $m < n$ hoặc nếu một số đặc trưng dư thừa, nhưng ma trận giả nghịch đảo luôn được xác định.

Độ phức tạp tính toán Phương trình chuẩn

tính toán ma trận nghịch đảo của XX^T , là một ma trận $(n + 1) \times (n + 1)$ (trong đó n là số lượng đặc trưng). Độ phức tạp tính toán của việc đảo ngược ma trận như vậy thường vào khoảng $O(n^3)$ đến $O(n^2.4)$, tùy thuộc vào cách triển khai. Nói cách khác, nếu bạn tăng gấp đôi số lượng đặc trưng, bạn sẽ nhân thời gian tính toán lên khoảng $2^3 = 8$ đến $2^{2.4} \approx 5.3$.

Phương pháp SVD được sử dụng bởi lớp `LinearRegression` của `Scikit-Learn` có độ phức tạp khoảng $O(n^2)$. Nếu bạn tăng gấp đôi số lượng đặc trưng, thời gian tính toán sẽ tăng lên khoảng 4 lần.

CẢNH BÁO

Cả phương pháp Phương trình Chuẩn và phương pháp SVD đều trở nên rất chậm khi số lượng đặc trưng tăng lên đáng kể (ví dụ: 100.000). Mặt khác, cả hai đều có độ phức tạp tuyến tính đối với số lượng mẫu trong tập huấn luyện (độ phức tạp $O(m)$), do đó chúng xử lý hiệu quả các tập huấn luyện lớn, miễn là chúng có thể nằm gọn trong bộ nhớ.

Ngoài ra, sau khi bạn đã huấn luyện mô hình Hồi quy tuyến tính (sử dụng Phương trình chuẩn hoặc bất kỳ thuật toán nào khác), quá trình dự đoán diễn ra rất nhanh: độ phức tạp tính toán tỷ lệ thuận với cả số lượng trường hợp bạn muốn dự đoán và số lượng đặc trưng. Nói cách khác, việc dự đoán trên số lượng trường hợp gấp đôi (hoặc số lượng đặc trưng gấp đôi) sẽ mất thời gian gấp đôi.

Bây giờ chúng ta sẽ xem xét một cách huấn luyện mô hình Hồi quy tuyến tính hoàn toàn khác, phù hợp hơn cho các trường hợp có số lượng đặc trưng lớn hoặc quá nhiều mẫu huấn luyện không thể chứa hết trong bộ nhớ.

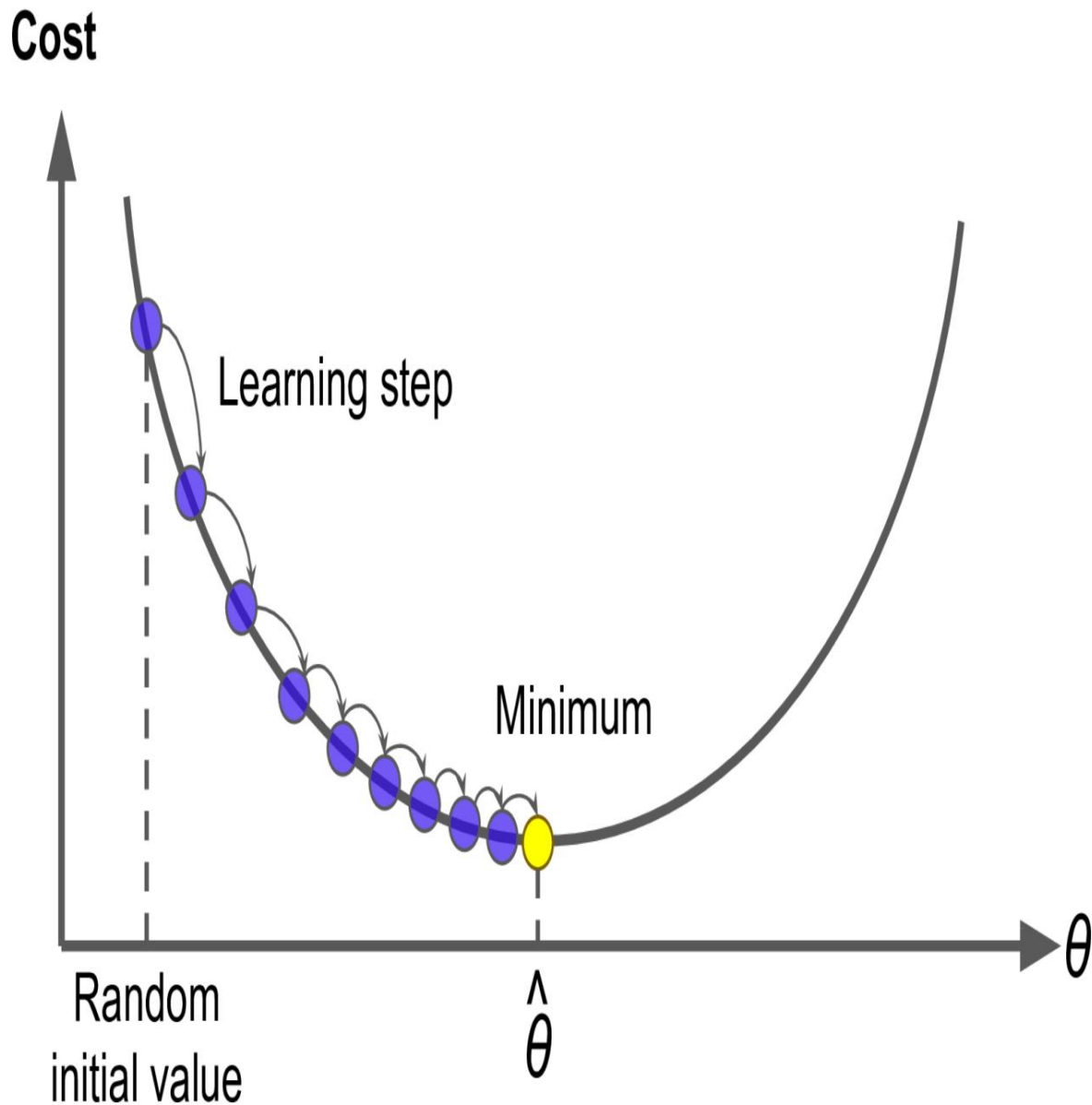
Giảm độ dốc

Thuật toán Gradient Descent là một thuật toán tối ưu hóa tổng quát có khả năng tìm ra các giải pháp tối ưu cho nhiều loại bài toán khác nhau. Ý tưởng chung của Gradient Descent là điều chỉnh các tham số một cách lặp đi lặp lại để giảm thiểu hàm chi phí.

Giả sử bạn bị lạc trong núi giữa một lớp sương mù dày đặc, và bạn chỉ có thể cảm nhận được độ dốc của mặt đất dưới chân mình. Một chiến lược tốt để nhanh chóng xuống đáy thung lũng là đi xuống dốc theo hướng độ dốc nhất.

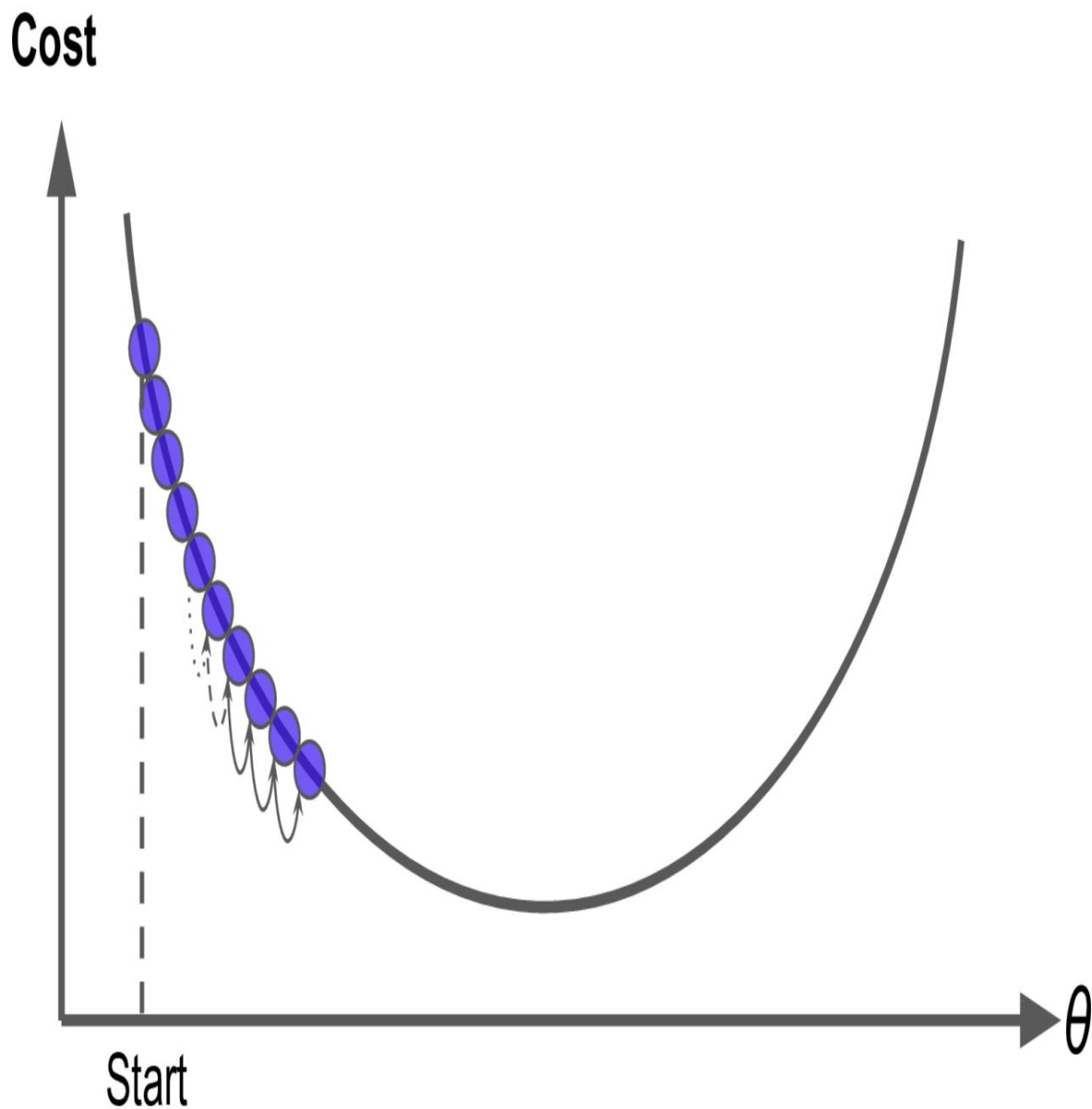
Độ dốc. Đây chính xác là những gì thuật toán Gradient Descent thực hiện: nó đo độ dốc cục bộ của hàm lỗi đối với vectơ tham số θ , và nó di chuyển theo hướng giảm độ dốc. Khi độ dốc bằng 0, bạn đã đạt đến điểm cực tiểu!

Trên thực tế, bạn bắt đầu bằng cách điền θ bằng các giá trị ngẫu nhiên (đây được gọi là khởi tạo ngẫu nhiên). Sau đó, bạn cải thiện nó dần dần, từng bước nhỏ một, mỗi bước cố gắng giảm hàm chi phí (ví dụ: MSE), cho đến khi thuật toán hội tụ về giá trị tối thiểu (xem [Hình 4-3](#)).



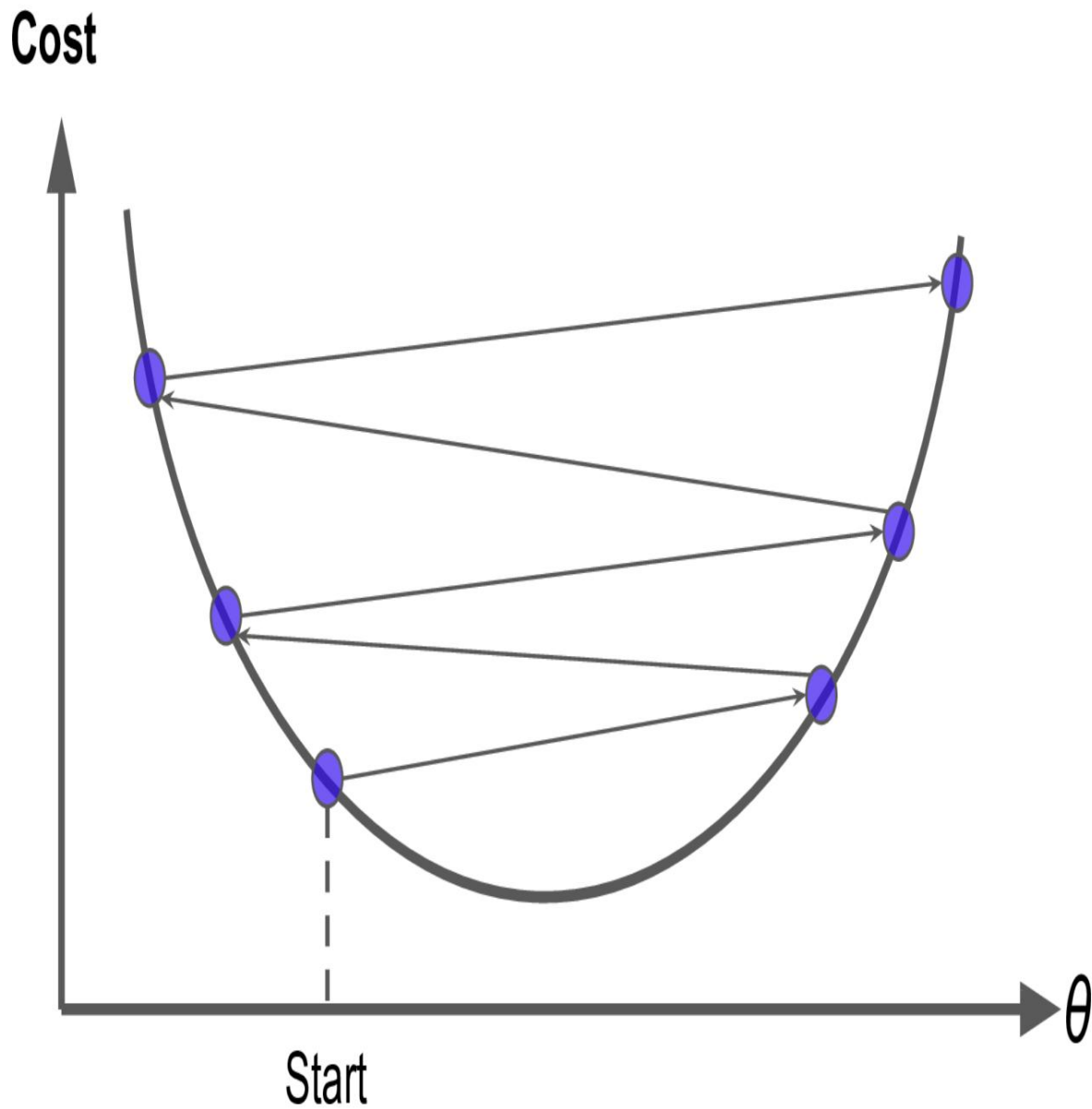
Hình 4-3. Trong hình minh họa về thuật toán Gradient Descent này, các tham số của mô hình được khởi tạo ngẫu nhiên và được điều chỉnh liên tục để giảm thiểu hàm chi phí; kích thước bước học tỷ lệ thuận với độ dốc của hàm chi phí, do đó các bước sẽ dần nhỏ lại khi chi phí tiến gần đến giá trị tối thiểu.

Một tham số quan trọng trong thuật toán Gradient Descent là kích thước của các bước, được xác định bởi siêu tham số tốc độ học. Nếu tốc độ học quá nhỏ, thuật toán sẽ phải trải qua nhiều lần lặp để hội tụ, điều này sẽ mất rất nhiều thời gian (xem [Hình 4-4](#)).



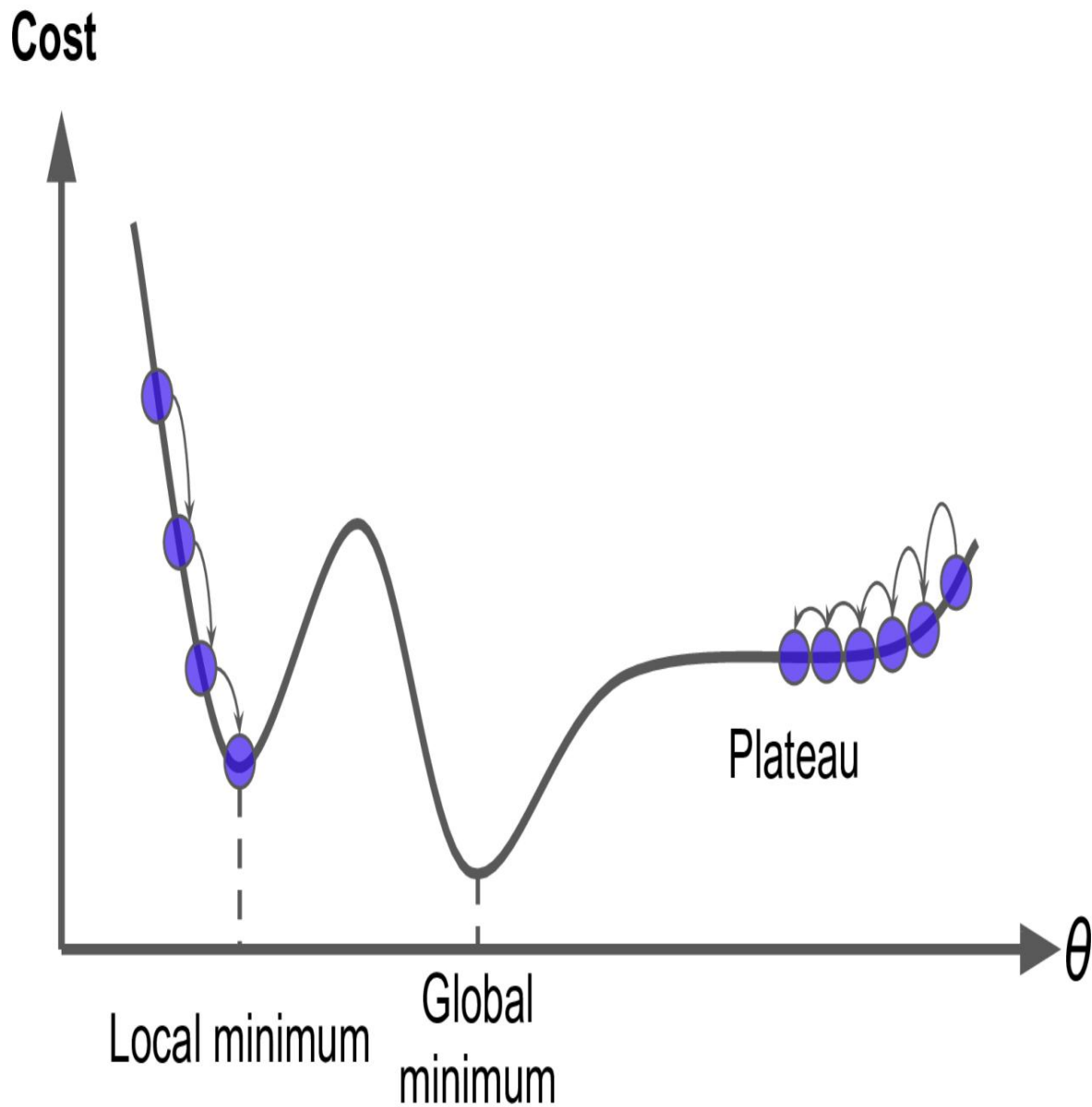
Hình 4-4. Tốc độ học quá nhỏ.

Mặt khác, nếu tốc độ học quá cao, bạn có thể nhảy qua thung lũng và kết thúc ở phía bên kia, thậm chí có thể ở vị trí cao hơn trước đó. Điều này có thể khiến thuật toán phân kỳ, với các giá trị ngày càng lớn, không tìm được giải pháp tốt (xem [Hình 4-5](#)).



Hình 4-5. Tốc độ học quá lớn.

Cuối cùng, không phải tất cả các hàm chi phí đều có hình dạng như những chiếc bát đều đặn, đẹp mắt. Có thể có những lỗ hổng, gờ, cao nguyên và đủ loại địa hình bất thường, khiến việc hội tụ đến điểm cực tiểu trở nên khó khăn. Hình 4-6 minh họa hai thách thức chính của thuật toán Gradient Descent. Nếu quá trình khởi tạo ngẫu nhiên bắt đầu thuật toán từ bên trái, thì nó sẽ hội tụ đến một điểm cực tiểu cục bộ, không tốt bằng điểm cực tiểu toàn cục. Nếu nó bắt đầu từ bên phải, thì sẽ mất rất nhiều thời gian để vượt qua cao nguyên. Và nếu bạn dừng quá sớm, bạn sẽ không bao giờ đạt được điểm cực tiểu toàn cục.

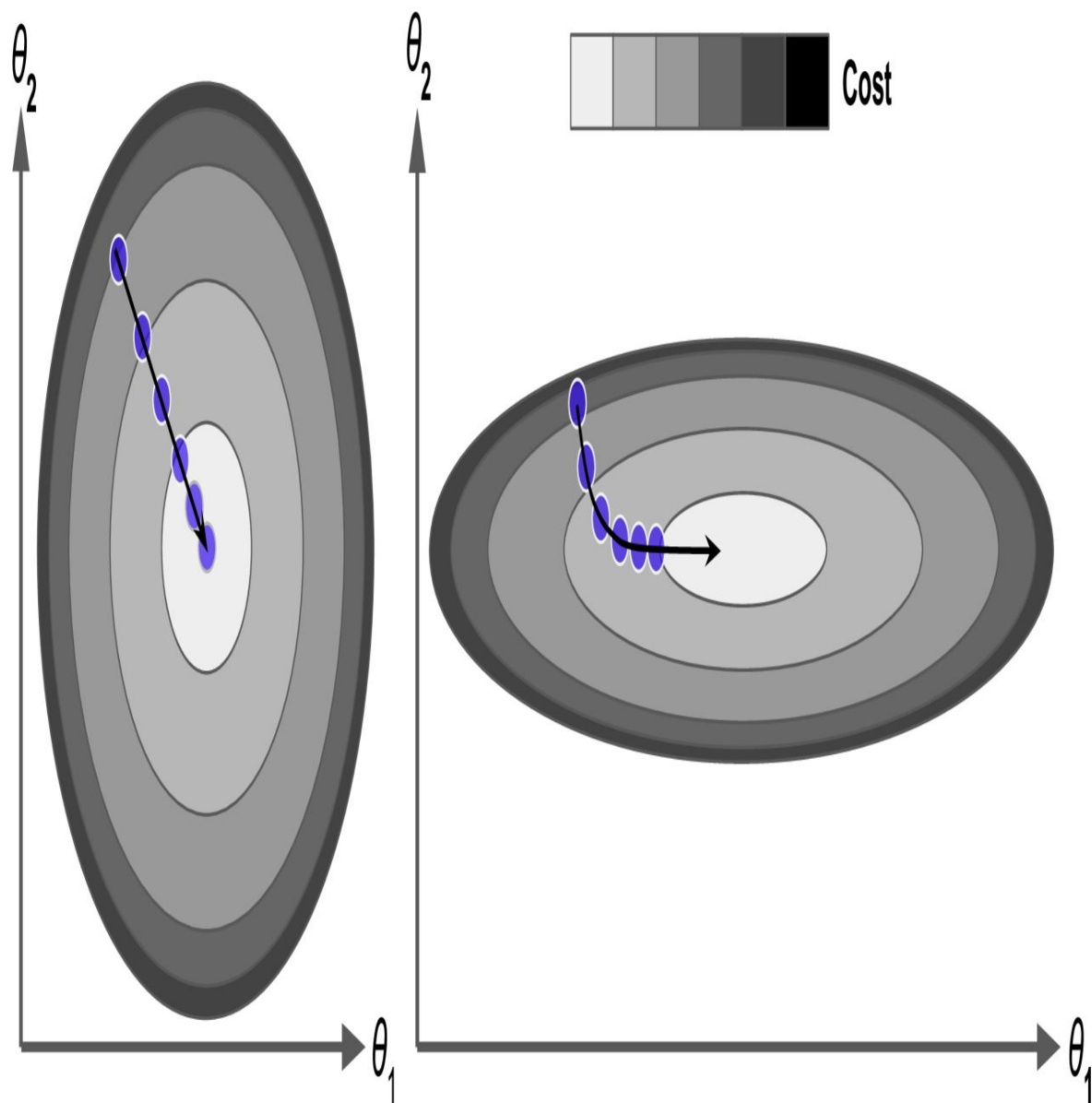


Hình 4-6. Những sai lầm thường gặp khi sử dụng thuật toán Gradient Descent.

May mắn thay, hàm chi phí MSE cho mô hình hồi quy tuyến tính là một hàm lồi, có nghĩa là nếu bạn chọn bất kỳ hai điểm nào trên đường cong, đoạn thẳng nối chúng sẽ không bao giờ nằm dưới đường cong.

Điều này ngụ ý rằng không có cực tiểu địa phương, chỉ có một cực tiểu toàn cục. Nó cũng là một hàm liên tục với độ dốc không bao giờ thay đổi đột ngột. Hai sự thật này có một hệ quả rất lớn: Thuật toán Gradient Descent được đảm bảo sẽ tiến đến rất gần cực tiểu toàn cục (nếu bạn chờ đủ lâu và nếu tốc độ học không quá cao).

Trên thực tế, hàm chi phí có hình dạng của một cái bát, nhưng nó có thể là một cái bát kéo dài nếu các đặc trưng có tỷ lệ khác nhau rất nhiều. Hình 4-7 minh họa thuật toán Gradient Descent trên tập huấn luyện trong đó các đặc trưng 1 và 2 có cùng tỷ lệ (bên trái), và trên tập huấn luyện trong đó đặc trưng 1 có giá trị nhỏ hơn nhiều so với đặc trưng 2 (bên phải).³



Hình 4-7. Thuật toán Gradient Descent có (trái) và không có (phải) việc điều chỉnh kích thước đặc trưng.

Như bạn thấy, ở bên trái, thuật toán Gradient Descent đi thẳng đến điểm cực tiểu, do đó đạt được điểm đó một cách nhanh chóng, trong khi ở bên phải, nó trước tiên đi theo hướng gần như vuông góc với hướng của điểm cực tiểu toàn cục, và kết thúc bằng một quá trình dài đi xuống một thung lũng gần như bằng phẳng. Cuối cùng nó cũng sẽ đạt đến điểm cực tiểu, nhưng sẽ mất rất nhiều thời gian.

CẢNH BÁO

Khi sử dụng Gradient Descent, bạn cần đảm bảo tất cả các đặc trưng có cùng thang đo (ví dụ: sử dụng lớp `StandardScaler` của `Scikit-Learn`), nếu không quá trình hội tụ sẽ mất nhiều thời gian hơn.

Sơ đồ này cũng minh họa thực tế rằng việc huấn luyện một mô hình có nghĩa là tìm kiếm sự kết hợp các tham số của mô hình sao cho hàm chi phí được tối thiểu hóa (trên tập dữ liệu huấn luyện). Đó là một quá trình tìm kiếm trong không gian tham số của mô hình: mô hình càng có nhiều tham số, không gian này càng có nhiều chiều, và quá trình tìm kiếm càng khó khăn hơn: tìm kiếm

Việc tìm một cây kim trong đống rơm 300 chiều khó hơn nhiều so với trong không gian 3 chiều. May mắn thay, vì hàm chi phí là hàm lồi trong trường hợp hồi quy tuyến tính, nên cây kim chỉ đơn giản nằm ở đáy đống rơm.

Giảm độ dốc theo lô

Để triển khai thuật toán Gradient Descent, bạn cần tính đạo hàm riêng của hàm chi phí đối với từng tham số mô hình θ_j và thay đổi θ một chút. Điều này được gọi là "độ dốc". Nói cách khác, bạn cần tính toán xem hàm chi phí sẽ thay đổi như thế nào nếu bạn... là đạo hàm riêng. Nó giống như việc hỏi "Độ dốc của ngọn núi dưới chân tôi là bao nhiêu nếu tôi quay mặt về hướng đông?" và sau đó hỏi cùng câu hỏi đó khi quay mặt về hướng bắc (và cứ thế cho tất cả các chiều khác, nếu bạn có thể tưởng tượng một vũ trụ có nhiều hơn ba chiều).

Phương trình 4-5 tính đạo hàm riêng của MSE đối với tham số θ .

θ_j , đã lưu ý $MSE(\theta) / \theta_j$.

Phương trình 4-5. Đạo hàm riêng của hàm chi phí

$$\frac{\partial}{\partial \theta_j} MSE(\theta) = \frac{2}{m} \sum_{i=1}^m (x^{(i)} - y^{(i)}) x_j^{(i)}$$

Thay vì tính toán các đạo hàm riêng lẻ này một cách riêng biệt, bạn có thể sử dụng Phương trình 4-6 để tính toán tất cả chúng cùng một lúc. Vectơ gradient, được ký hiệu là $\nabla_{\theta} MSE(\theta)$, chứa tất cả các đạo hàm riêng của hàm chi phí (một cho mỗi tham số mô hình).

Phương trình 4-6. Vectơ gradient của hàm chi phí

$$\nabla_{\theta} MSE(\theta) = \begin{bmatrix} \frac{\partial}{\partial \theta_0} MSE(\theta) \\ \frac{\partial}{\partial \theta_1} MSE(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} MSE(\theta) \end{bmatrix} = \begin{bmatrix} \frac{2}{m} \sum_{i=1}^m (x^{(i)} - y^{(i)}) \\ \frac{2}{m} \sum_{i=1}^m (x^{(i)} - y^{(i)}) x_1^{(i)} \\ \vdots \\ \frac{2}{m} \sum_{i=1}^m (x^{(i)} - y^{(i)}) x_n^{(i)} \end{bmatrix}$$

CẢNH BÁO

Hãy lưu ý rằng công thức này bao gồm các phép tính trên toàn bộ tập dữ liệu huấn luyện X , ở mỗi bước của thuật toán Gradient Descent! Đó là lý do tại sao thuật toán này được gọi là Batch Gradient Descent: nó sử dụng toàn bộ tập dữ liệu huấn luyện ở mỗi bước (thực ra, Full Gradient Descent có lẽ là một cái tên phù hợp hơn). Kết quả là nó hoạt động cực kỳ chậm trên các tập dữ liệu huấn luyện rất lớn (nhưng chúng ta sẽ thấy các thuật toán Gradient Descent nhanh hơn nhiều trong thời gian ngắn tới). Tuy nhiên, Gradient Descent có khả năng mở rộng tốt với số lượng đặc trưng; việc huấn luyện một mô hình hồi quy tuyến tính khi có hàng trăm nghìn đặc trưng sẽ nhanh hơn nhiều khi sử dụng Gradient Descent so với việc sử dụng Phương trình chuẩn hoặc phân rã SVD.

Khi bạn đã có vectơ độ dốc, hướng lên dốc, chỉ cần đi theo hướng ngược lại để xuống dốc. Điều này có nghĩa là trừ $MSE(\theta)$ khỏi θ . Đây là ⁴đức tốc độ học η phát huy tác dụng: nhân vectơ độ dốc với η để xác định kích thước của bước xuống dốc (Phương trình 4-7).

Phương trình 4-7. Bước giảm độ dốc

$$\theta = \theta - \eta \nabla_{\theta} MSE(\theta)$$

Hãy cùng xem một ví dụ triển khai nhanh thuật toán này:

```
eta = 0.1 # tốc độ học
n_epochs = 1000
m = len(X_b) # số lượng mẫu

np.random.seed(42)
theta = np.random.randn(2, 1) # các tham số mô hình được khởi tạo ngẫu nhiên
```

```

for epoch in range(n_epochs): gradients
    = 2 / m * X_b.T @ (X_b @ theta - y) * gradients
    theta = theta - eta

```

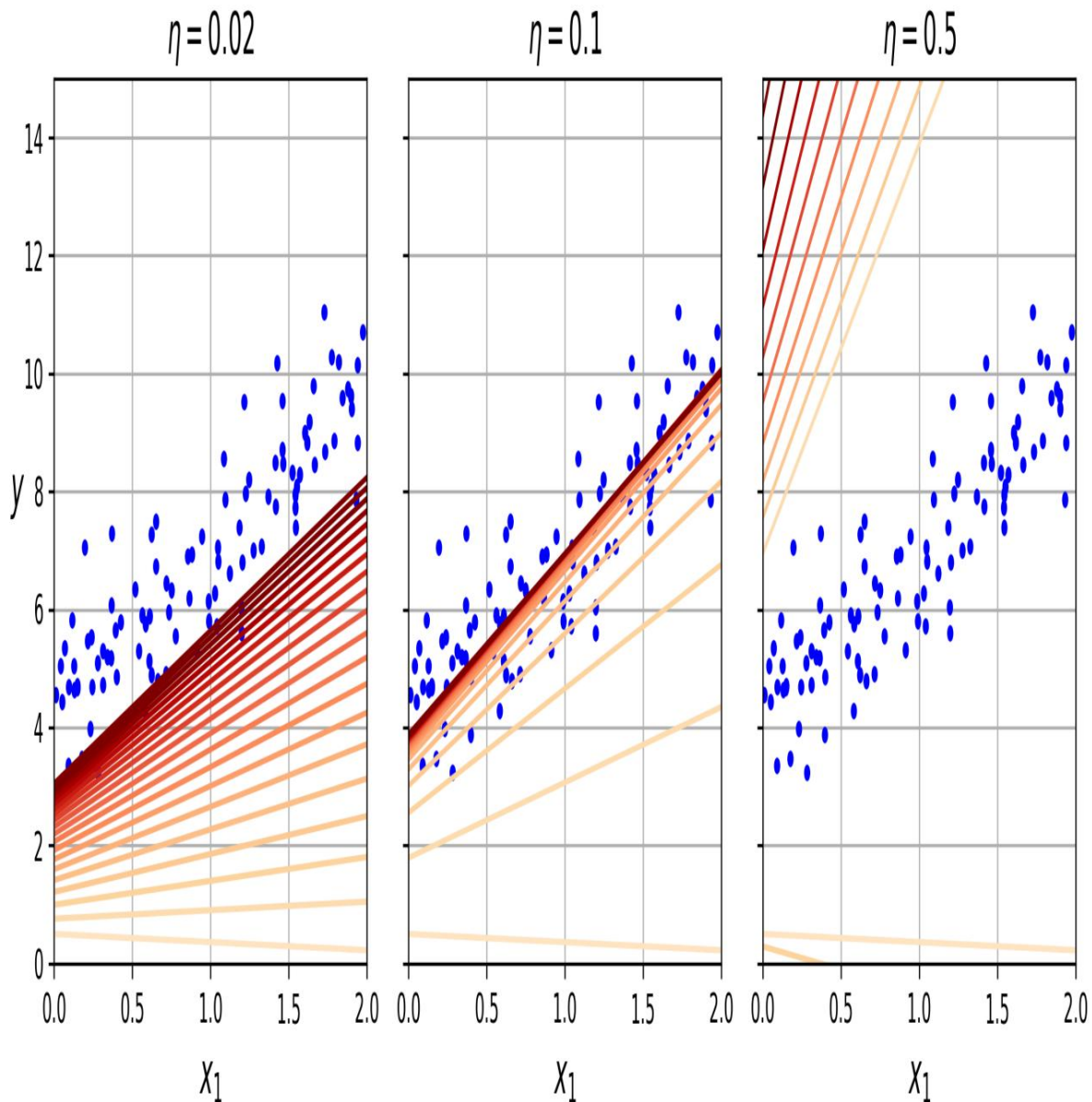
Việc này không quá khó! Mỗi lần lặp lại trên tập dữ liệu huấn luyện được gọi là một epoch. Hãy cùng xem kết quả theta:

```

>>> theta
array([[4.21509616],
       [2.77011339]])

```

Này, đó chính xác là những gì phương trình chuẩn đã tìm ra! Thuật toán Gradient Descent hoạt động hoàn hảo. Nhưng nếu bạn sử dụng tốc độ học khác eta thì sao? Hình 4-8 hiển thị 20 bước đầu tiên của thuật toán Gradient Descent sử dụng ba tốc độ học khác nhau. Đường ở dưới cùng của mỗi đồ thị biểu thị điểm bắt đầu ngẫu nhiên, sau đó mỗi epoch được biểu thị bằng một đường ngày càng đậm hơn.



Hình 4-8. Thuật toán Gradient Descent với các tốc độ học khác nhau.

Ở bên trái, tốc độ học quá thấp: thuật toán cuối cùng sẽ đạt được lời giải, nhưng sẽ mất rất nhiều thời gian. Ở giữa, tốc độ học trông khá tốt: chỉ sau vài epoch, nó đã hội tụ đến lời giải. Ở bên phải, tốc độ học quá cao: thuật toán phân kỳ, nhảy lung tung và thực tế ngày càng xa lời giải hơn ở mỗi bước.

Để tìm được tốc độ học phù hợp, bạn có thể sử dụng phương pháp tìm kiếm lưới (xem [Chương 2](#)). Tuy nhiên, bạn có thể muốn giới hạn số lượng epoch để phương pháp tìm kiếm lưới có thể loại bỏ những mô hình mất quá nhiều thời gian để hội tụ.

Bạn có thể thắc mắc làm thế nào để thiết lập số lượng epoch. Nếu quá ít, bạn vẫn sẽ cách xa giải pháp tối ưu khi thuật toán dừng lại; nhưng nếu quá nhiều, bạn sẽ lãng phí thời gian trong khi các tham số của mô hình không thay đổi nữa. Một giải pháp đơn giản là thiết lập số lượng epoch rất lớn nhưng dừng thuật toán khi vectơ gradient trở nên rất nhỏ – tức là khi chuẩn của nó nhỏ hơn một giá trị ϵ (được gọi là dung sai rất nhỏ – bởi vì điều này xảy ra khi Gradient Descent đã (gần như) đạt đến điểm cực tiểu).

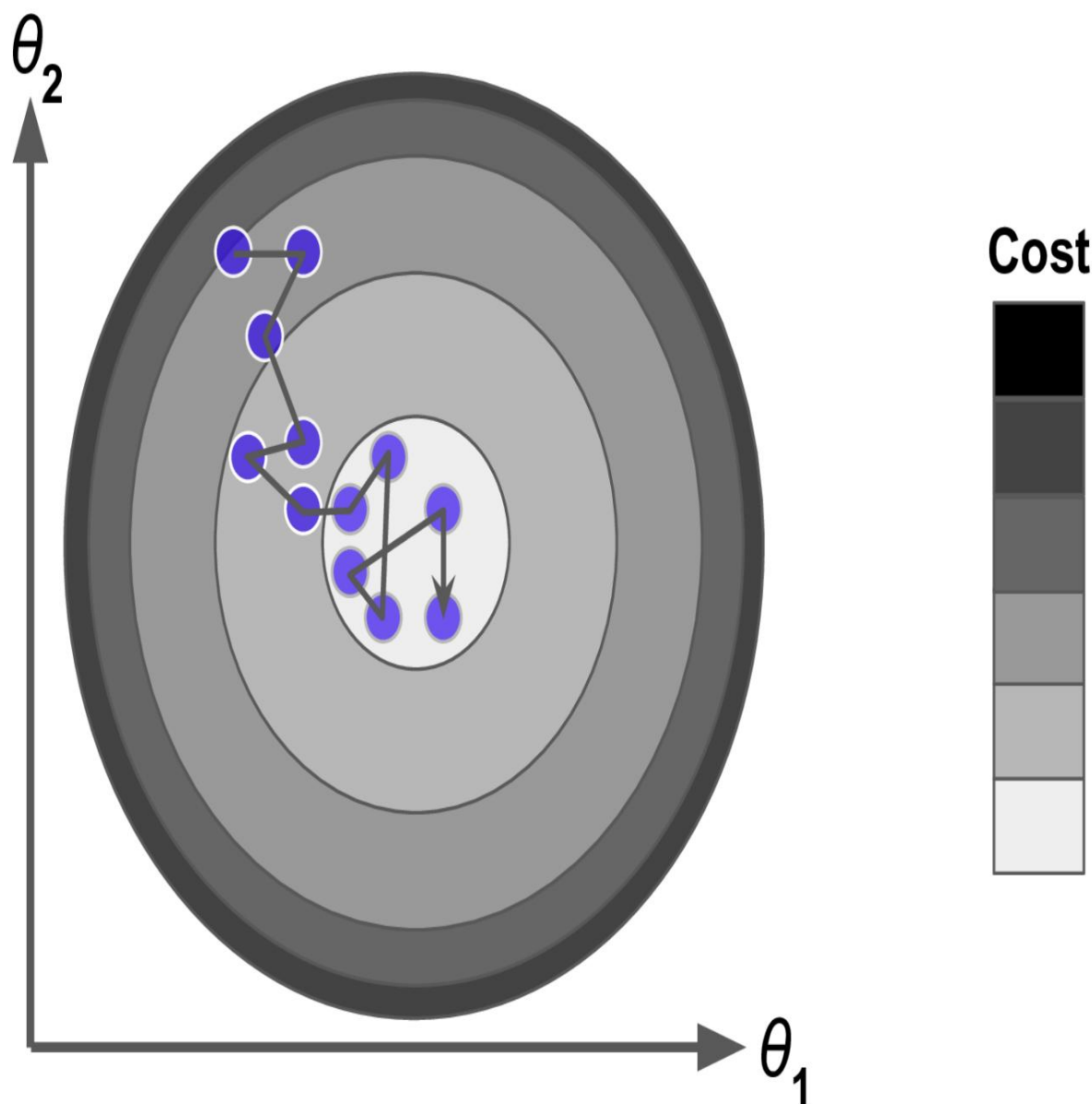
TỐC ĐỘ HỘI TỤ

Khi hàm chi phí lồi và độ dốc của nó không thay đổi đột ngột (như trường hợp của hàm chi phí MSE), thuật toán Batch Gradient Descent với tốc độ học cố định cuối cùng sẽ hội tụ đến giải pháp tối ưu, nhưng bạn có thể phải đợi một thời gian: có thể mất $O(1/\epsilon)$ lần lặp để đạt được giá trị tối ưu trong một ϵ phạm vi, tùy thuộc vào hình dạng của hàm chi phí. Nếu bạn chia dung sai cho 10 để có giải pháp chính xác hơn, thì thuật toán có thể phải chạy lâu hơn khoảng 10 lần.

Thuật toán giảm độ dốc ngẫu nhiên (Stochastic Gradient Descent)

Vấn đề chính của thuật toán Batch Gradient Descent là nó sử dụng toàn bộ tập dữ liệu huấn luyện để tính toán gradient ở mỗi bước, điều này khiến thuật toán rất chậm khi tập dữ liệu huấn luyện lớn. Ngược lại, thuật toán Stochastic Gradient Descent chọn ngẫu nhiên một trường hợp trong tập dữ liệu huấn luyện ở mỗi bước và tính toán gradient chỉ dựa trên trường hợp đó. Rõ ràng, việc xử lý từng trường hợp một giúp thuật toán nhanh hơn nhiều vì nó chỉ cần xử lý rất ít dữ liệu ở mỗi lần lặp. Điều này cũng cho phép huấn luyện trên các tập dữ liệu khổng lồ, vì chỉ cần một trường hợp nằm trong bộ nhớ ở mỗi lần lặp (Stochastic Gradient Descent có thể được triển khai như một thuật toán xử lý ngoài bộ nhớ; xem [Chương 1](#)).

Mặt khác, do bản chất ngẫu nhiên (tức là không có quy luật), thuật toán này ít đều đặn hơn nhiều so với thuật toán Giảm độ dốc theo lô: thay vì giảm dần cho đến khi đạt mức tối thiểu, hàm chi phí sẽ dao động lên xuống, chỉ giảm trung bình. Theo thời gian, nó sẽ tiến rất gần đến mức tối thiểu, nhưng một khi đạt đến đó, nó sẽ tiếp tục dao động xung quanh, không bao giờ ổn định (xem [Hình 4-9](#)). Vì vậy, một khi thuật toán dừng lại, các giá trị tham số cuối cùng là tốt, nhưng không tối ưu.



Hình 4-9. Với thuật toán Gradient Descent ngẫu nhiên (Stochastic Gradient Descent), mỗi bước huấn luyện diễn ra nhanh hơn nhiều nhưng cũng mang tính ngẫu nhiên cao hơn so với khi sử dụng thuật toán Gradient Descent theo lô (Batch Gradient Descent).

Khi hàm chi phí rất bất thường (như trong Hình 4-6), điều này thực sự có thể giúp thuật toán thoát khỏi các cực tiểu cục bộ, do đó thuật toán Gradient Descent ngẫu nhiên có cơ hội tìm thấy cực tiểu toàn cục tốt hơn so với thuật toán Gradient Descent theo lô.

Do đó, tính ngẫu nhiên rất tốt để thoát khỏi các điểm tối ưu cục bộ, nhưng lại không tốt vì nó có nghĩa là thuật toán không bao giờ có thể ổn định ở điểm cực tiểu. Một giải pháp cho vấn đề này là giảm dần tốc độ học. Các bước giảm ban đầu lớn (giúp tiến bộ nhanh và thoát khỏi các điểm cực tiểu cục bộ), sau đó nhỏ dần, cho phép thuật toán ổn định ở điểm cực tiểu toàn cục. Quá trình này tương tự như thuật toán luyện kim mô phỏng (simulated annealing), một thuật toán lấy cảm hứng từ quá trình luyện kim, trong đó kim loại nóng chảy được làm nguội từ từ. Hàm xác định tốc độ học ở mỗi lần lặp được gọi là lịch trình học (learning schedule). Nếu tốc độ học giảm quá nhanh, bạn có thể bị mắc kẹt ở điểm cực tiểu cục bộ, hoặc thậm chí bị đóng băng ở giữa chừng đến điểm cực tiểu. Nếu tốc độ học giảm quá chậm, bạn có thể nhảy xung quanh điểm cực tiểu trong một thời gian dài và kết thúc với một giải pháp không tối ưu nếu bạn dừng huấn luyện quá sớm.

Đoạn mã này triển khai thuật toán Giảm độ dốc ngẫu nhiên (Stochastic Gradient Descent) sử dụng một lịch trình học tập đơn giản:

```
n_epochs = 50 t0,
t1 = 5, 50 # tham số siêu tham số của lịch trình học

def learning_schedule(t): return
    t0 / (t + t1)

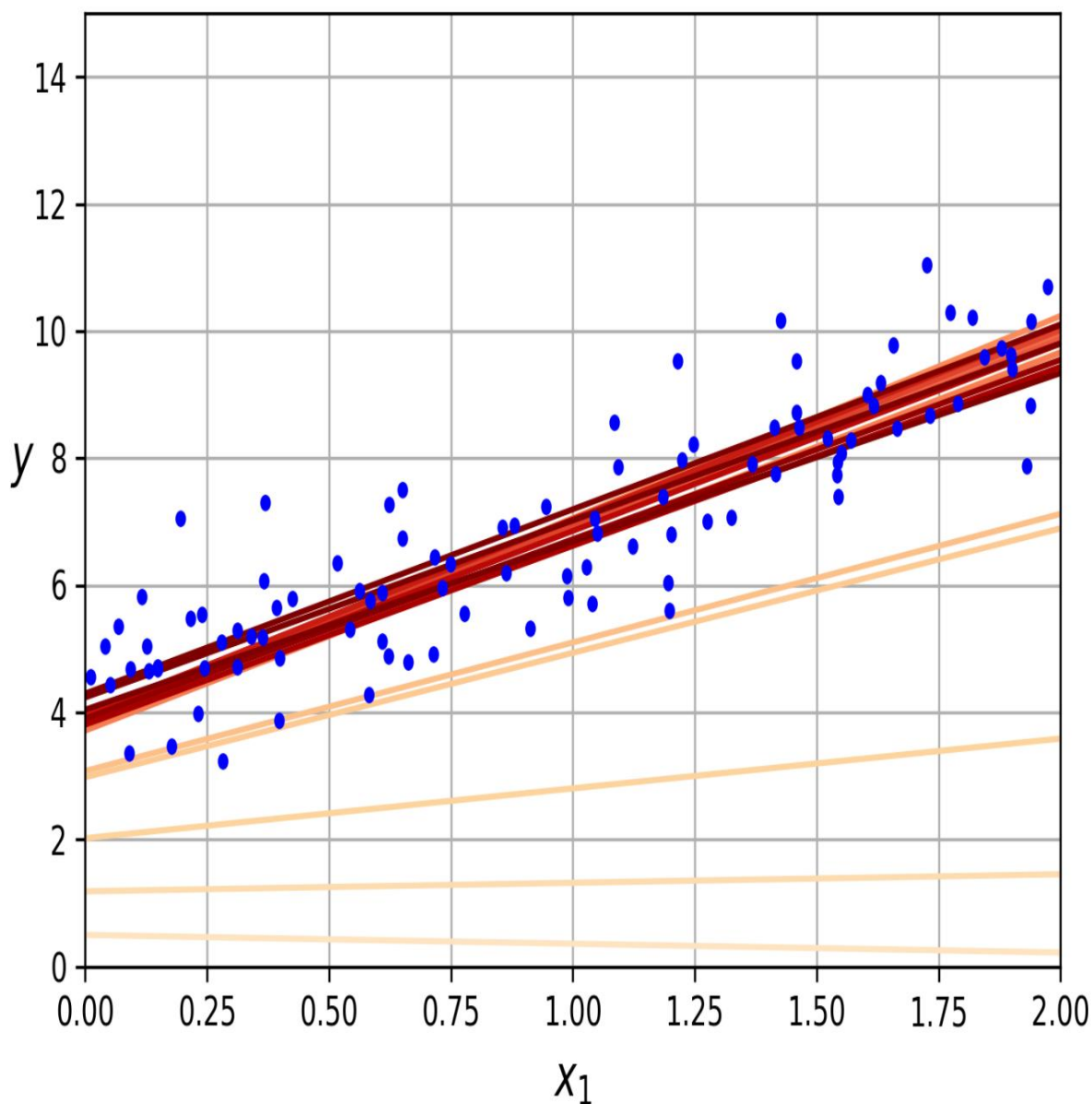
np.random.seed(42) theta
= np.random.randn(2, 1) # khởi tạo ngẫu nhiên

for epoch in range(n_epochs): for iteration in
    range(m): random_index = np.random.randint(m)
        xi = X_b[random_index : random_index + 1] yi =
        y[random_index : random_index + 1] gradients = 2 * xi.T @ (xi @
        theta - yi) # đối với SGD, không chia cho m eta =
        learning_schedule(epoch * m + iteration) theta = theta - eta * gradients
```

Theo quy ước, chúng ta lặp lại theo từng vòng với m lần lặp; mỗi vòng được gọi là một epoch, như đã đề cập trước đó. Trong khi mã Batch Gradient Descent lặp lại 1.000 lần trên toàn bộ tập dữ liệu huấn luyện, mã này chỉ lặp lại 50 lần trên tập dữ liệu huấn luyện và đạt được một giải pháp khá tốt:

```
>>> theta
array([[4.21076011],
       [2.74856079]])
```

Hình 4-10 minh họa 20 bước đầu tiên của quá trình huấn luyện (hãy chú ý các bước này không đều nhau như thế nào).



Hình 4-10. 20 bước đầu tiên của thuật toán Giảm độ dốc ngẫu nhiên (Stochastic Gradient Descent).

Lưu ý rằng vì các trường hợp được chọn ngẫu nhiên, một số trường hợp có thể được chọn nhiều lần trong mỗi epoch, trong khi những trường hợp khác có thể không được chọn. Nếu bạn muốn đảm bảo rằng thuật toán xử lý mọi trường hợp trong mỗi epoch, một cách tiếp cận khác là xáo trộn tập dữ liệu huấn luyện (đảm bảo xáo trộn đồng thời các đặc trưng đầu vào và nhãn), sau đó xử lý từng trường hợp một, rồi lại xáo trộn lần nữa, và cứ thế tiếp tục. Tuy nhiên, cách tiếp cận này phức tạp hơn và nhìn chung không cải thiện được kết quả.

CẢNH BÁO

Khi sử dụng thuật toán Stochastic Gradient Descent (SGD), các mẫu huấn luyện phải độc lập và phân bố đồng nhất (IID) để đảm bảo các tham số được kéo về phía điểm tối ưu toàn cục, trung bình. Một cách đơn giản để đảm bảo điều này là xáo trộn các mẫu trong quá trình huấn luyện (ví dụ: chọn ngẫu nhiên từng mẫu hoặc xáo trộn tập huấn luyện ở đầu mỗi epoch). Nếu bạn không xáo trộn các mẫu-ví dụ: nếu các mẫu được sắp xếp theo nhãn-thì SGD sẽ bắt đầu bằng cách tối ưu hóa cho một nhãn, sau đó là nhãn tiếp theo, v.v., và nó sẽ không ổn định gần điểm cực tiểu toàn cục.

Để thực hiện Hồi quy tuyến tính bằng Stochastic GD với Scikit-Learn, bạn có thể sử dụng lớp `SGDRegressor`, mặc định tối ưu hóa hàm chi phí MSE. Đoạn mã sau chạy tối đa 1.000 epoch (`max_iter`) hoặc cho đến khi tổn thất giảm xuống dưới 10 (`tol`) trong 100 epoch (`n_iter_no_change`). Nó bắt đầu với tốc độ học là 0,01 (`eta0`), sử dụng lịch trình học mặc định (khác với lịch trình chúng ta đã sử dụng).

Cuối cùng, nó không sử dụng bất kỳ phương pháp điều chỉnh nào (`penalty=None`; chi tiết hơn về điều này sẽ được trình bày sau):

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None, eta0=0.01,
                       n_iter_no_change=100, random_state=42)
sgd_reg.fit(X, y.ravel()) # y.ravel() vì fit() mong đợi mục tiêu 1D
```

Một lần nữa, bạn lại tìm thấy một nghiệm khá gần với nghiệm thu được từ Phương trình Chuẩn:

```
>>> sgd_reg.intercept_, sgd_reg.coef_
(array([4.21278812]), array([2.77270267]))
```

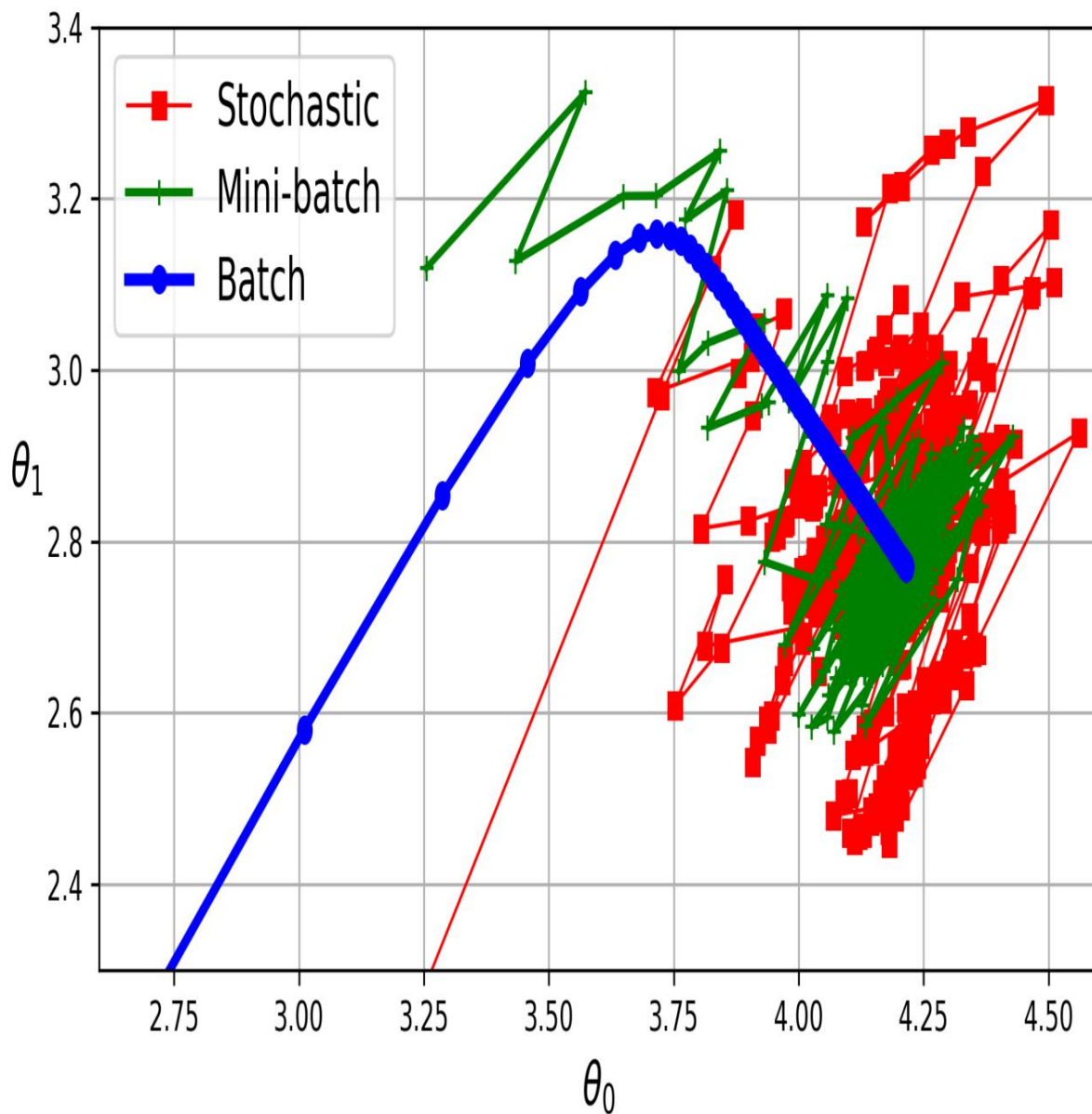
MẸO

Tất cả các mô hình ước lượng của Scikit-Learn đều có thể được huấn luyện bằng phương thức `'fit()'`, nhưng một số mô hình cũng có phương thức `'partial_fit()'` mà bạn có thể gọi để chạy một vòng huấn luyện duy nhất trên một hoặc nhiều trường hợp (nó bỏ qua các siêu tham số như `'max_iter'` hoặc `'tol'`). Việc gọi `'partial_fit()'` nhiều lần sẽ huấn luyện mô hình một cách dần dần. Điều này hữu ích khi bạn cần kiểm soát nhiều hơn quá trình huấn luyện. Các mô hình khác có siêu tham số `'warm_start'` (và một số có cả hai): nếu bạn đặt `'warm_start=True'`, việc gọi phương thức `'fit()'` trên một mô hình đã được huấn luyện sẽ không đặt lại mô hình, nó chỉ tiếp tục huấn luyện từ nơi nó đã dừng lại, tôn trọng các siêu tham số như `'max_iter'` và `'tol'`. Lưu ý rằng `'fit()'` đặt lại bộ đếm số lần lặp được sử dụng bởi lịch trình học, trong khi `'partial_fit()'` thì không.

Giảm độ dốc theo lô nhỏ

Thuật toán Gradient Descent cuối cùng mà chúng ta sẽ xem xét được gọi là Mini-batch Gradient Descent. Nó khá đơn giản một khi bạn đã biết về Batch và Stochastic Gradient Descent: ở mỗi bước, thay vì tính toán gradient dựa trên toàn bộ tập dữ liệu huấn luyện (như trong Batch GD) hoặc chỉ dựa trên một trường hợp duy nhất (như trong Stochastic GD), Mini-batch GD tính toán gradient trên các tập hợp nhỏ ngẫu nhiên các trường hợp được gọi là mini-batch. Ưu điểm chính của Mini-batch GD so với Stochastic GD là bạn có thể tăng hiệu suất nhờ tối ưu hóa phần cứng của các phép toán ma trận, đặc biệt khi sử dụng Bộ xử lý đồ họa (GPU).

Quá trình tiến triển của thuật toán trong không gian tham số ít thất thường hơn so với Stochastic GD, đặc biệt là với các mini-batch khá lớn. Kết quả là, Mini-batch GD sẽ di chuyển gần điểm cực tiểu hơn so với Stochastic GD—nhưng nó có thể khó thoát khỏi các cực tiểu cục bộ hơn (trong trường hợp các bài toán bị cực tiểu cục bộ, không giống như Hồi quy tuyến tính với hàm chi phí MSE). Hình 4-11 cho thấy các đường đi của ba thuật toán Gradient Descent trong không gian tham số trong quá trình huấn luyện. Tất cả đều kết thúc gần điểm cực tiểu, nhưng đường đi của Batch GD thực sự dừng lại ở điểm cực tiểu, trong khi cả Stochastic GD và Mini-batch GD đều tiếp tục di chuyển xung quanh. Tuy nhiên, đừng quên rằng Batch GD mất rất nhiều thời gian để thực hiện mỗi bước, và Stochastic GD và Mini-batch GD cũng sẽ đạt đến điểm cực tiểu nếu bạn sử dụng lịch trình học tập tốt.



Hình 4-11. Đường dẫn giảm độ dốc trong không gian tham số

Hãy so sánh các thuật toán mà chúng ta đã thảo luận cho đến nay về Hồi quy tuyến tính (nhớ rằng m là số lượng mẫu huấn luyện và n là số lượng đặc trưng); xem [Bảng 4-1](#).

T
Một
b
le
4
-
1
.
C
o
m
P
Một
ri
S
o
N
o
f
Một
l
g
o
ri
t
h
m
S
f
o
r
L
...
N
e
Một
r
R
e
g
r
e
s
si
o
N

Thuật toán	Lớn m	Hỗ trợ ngoài bộ nhớ chính	Số lớn	Siêu tham số cần thiết để điều chỉnh tỷ lệ trong Scikit-Learn
------------	-------	------------------------------	--------	---

Phương trình chuẩn	Nhanh	KHÔNG	Chậm	0	KHÔNG	không dự đoán
SVD	Nhanh	KHÔNG	Chậm	0	KHÔNG	Hồi quy tuyến tính ion
Lô GD	Chậm	KHÔNG	Nhanh	2	Đúng	không dự đoán
GD ngẫu nhiên nhanh		Đúng	Nhanh	≥2	Đúng	SGDRegressor
GD Fast theo lô nhỏ		Đúng	Nhanh	≥2	Đúng	không dự đoán

Hầu như không có sự khác biệt nào sau quá trình huấn luyện: tất cả các thuật toán này đều tạo ra các mô hình rất giống nhau và giúp tạo ra...

Các dự đoán cũng theo cách hoàn toàn tương tự.

Hồi quy đa thức

Điều gì sẽ xảy ra nếu dữ liệu của bạn phức tạp hơn một đường thẳng? Thật ngạc nhiên, bạn có thể sử dụng mô hình tuyến tính để phù hợp với dữ liệu phi tuyến tính.

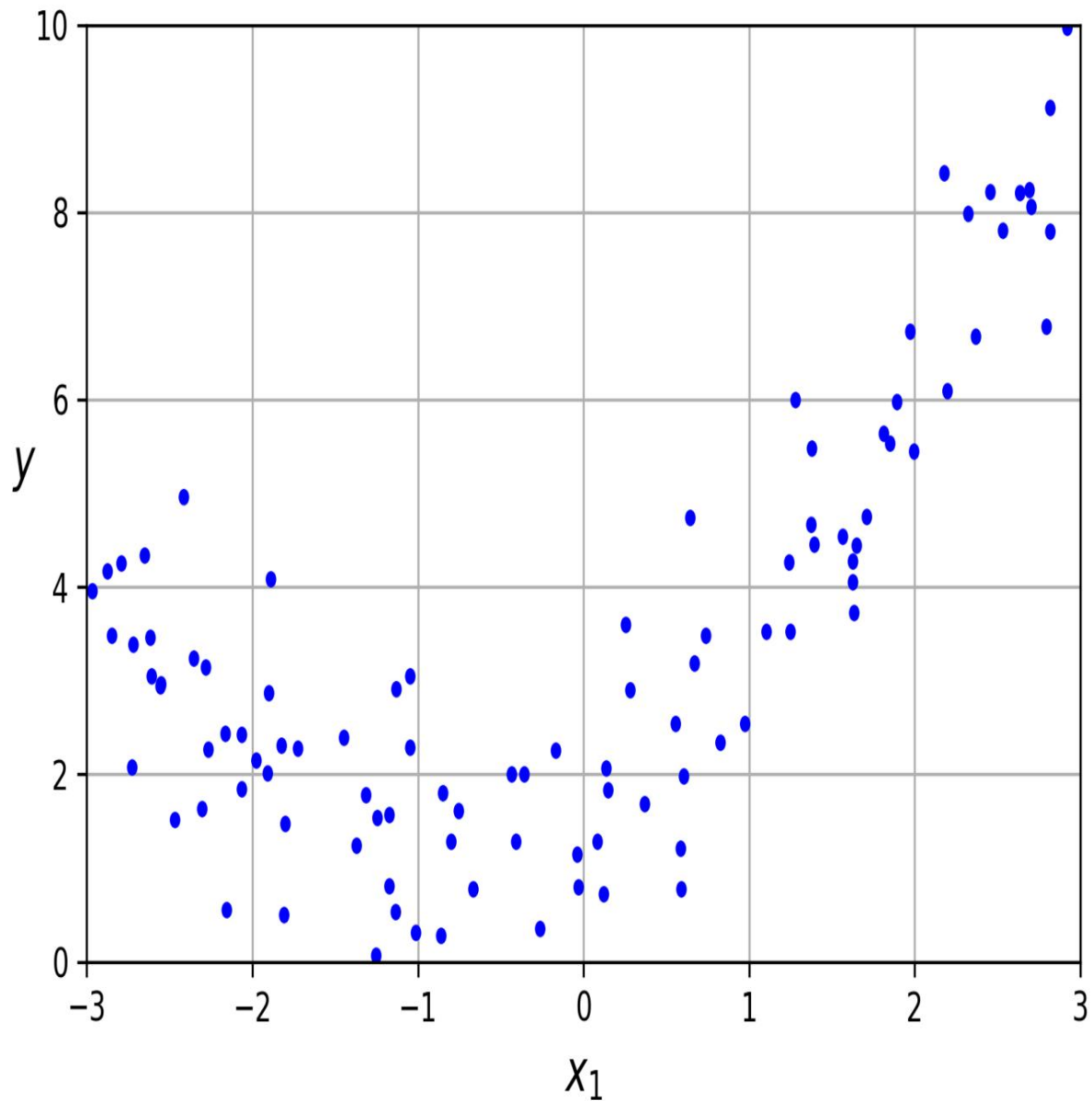
dữ liệu. Một cách đơn giản để làm điều này là thêm lũy thừa của mỗi đặc trưng dưới dạng các đặc trưng mới, sau đó huấn luyện một mô hình tuyến tính trên đó.

Tập hợp các tính năng mở rộng. Kỹ thuật này được gọi là Hồi quy đa thức.

Hãy xem một ví dụ. Đầu tiên, chúng ta hãy tạo ra một số dữ liệu phi tuyến tính (xem [Hình 4-12](#)), dựa trên một công thức đơn giản.

Phương trình bậc hai – đó là phương trình có dạng $y = ax^2 + bx + c$ – cộng thêm một số nhiễu:

```
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X**2 + X + 2 * np.random.randn(m, 1)
```



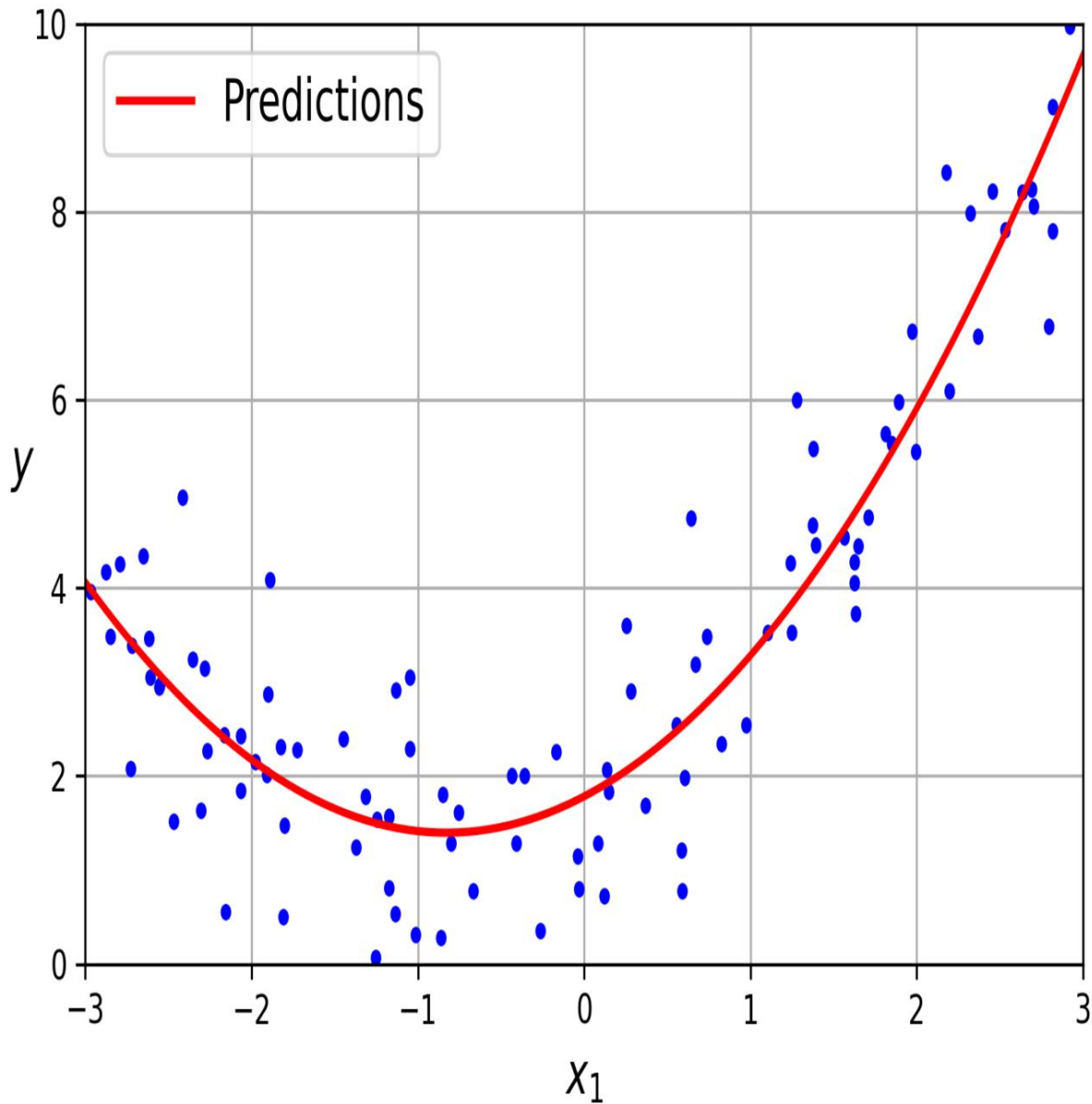
Hình 4-12. Tập dữ liệu phi tuyến tính và nhiễu được tạo ra.

Rõ ràng, một đường thẳng sẽ không bao giờ phù hợp với dữ liệu này một cách chính xác. Vì vậy, chúng ta hãy sử dụng lớp `PolynomialFeatures` của Scikit-Learn để biến đổi dữ liệu huấn luyện của mình, bằng cách thêm bình phương (đa thức bậc hai) của mỗi đặc trưng trong tập huấn luyện làm đặc trưng mới (trong trường hợp này chỉ có một đặc trưng):

```
>>> from sklearn.preprocessing import PolynomialFeatures >>>
poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929, 0.56664654])
```

`X_poly` hiện chứa đặc trưng gốc của `X` cộng với bình phương của đặc trưng này. Giờ đây, bạn có thể áp dụng mô hình Hồi quy tuyến tính cho dữ liệu huấn luyện mở rộng này (Hình 4-13):

```
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```



Hình 4-13. Dự đoán của mô hình hồi quy đa thức

Không tệ: mô hình ước tính $\hat{y} = 0,56x_1^2 + 0,93x_1 + 1,78$ trong khi thực tế hàm số ban đầu là $y = 0,5x_1^2 + 1,0x_1 + 2,0 + \text{Nhiều Gaussian}$.

Lưu ý rằng khi có nhiều đặc trưng, hồi quy đa thức có khả năng tìm ra mối quan hệ giữa chúng.

các tính năng mà mô hình hồi quy tuyến tính thông thường không thể thực hiện được. Điều này có thể thực hiện được nhờ vào thực tế là PolynomialFeatures cũng cộng tất cả các tổ hợp đặc trưng lên đến bậc đã cho. Ví dụ, nếu có

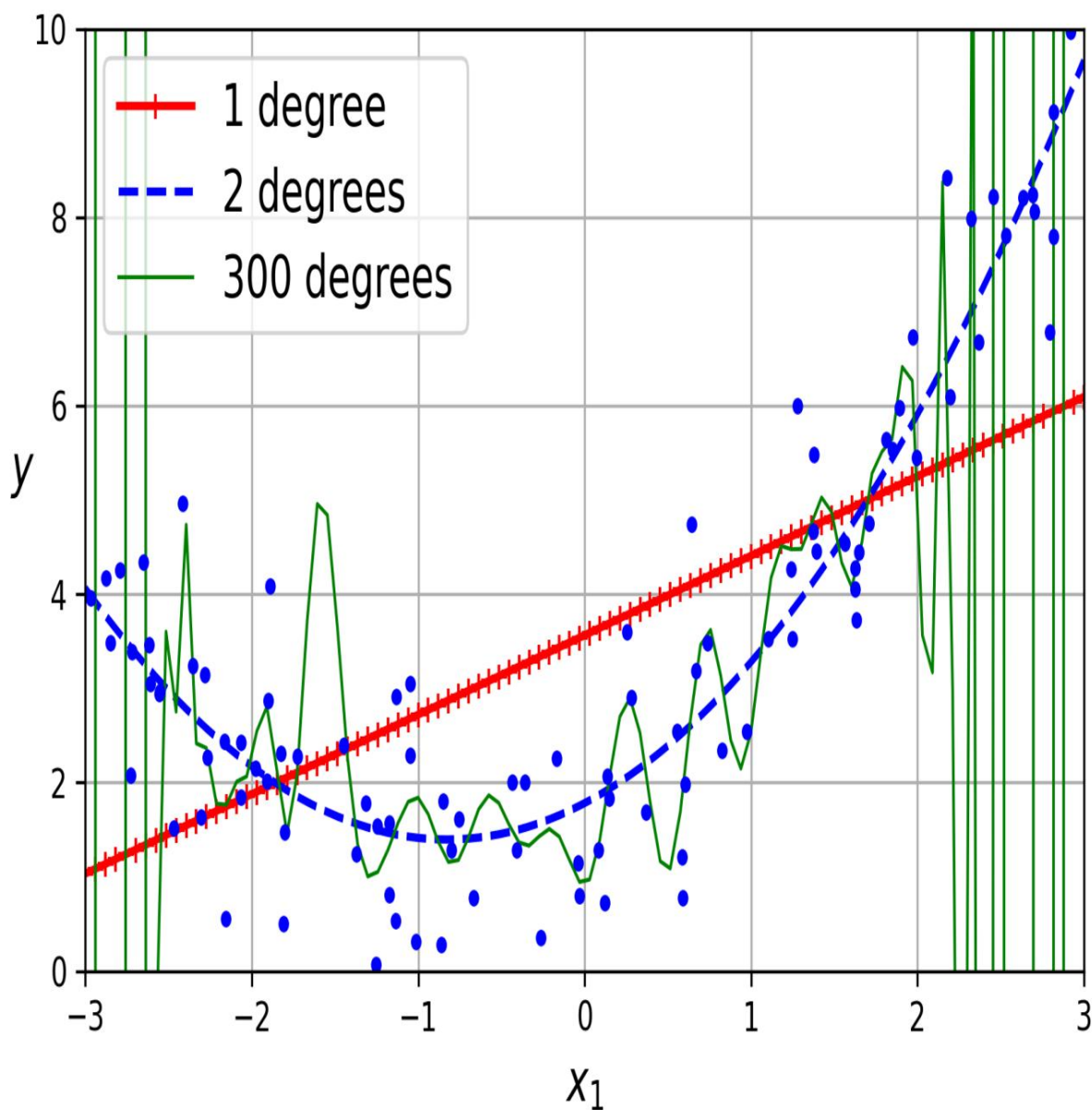
Nếu có hai đặc trưng a và b , thì PolynomialFeatures với bậc = 3 sẽ không chỉ cộng các đặc trưng ab mà còn cộng thêm các đặc trưng a^2b , ab^2 , và b^3 , nhưng cũng bao gồm các tổ hợp ab^2 , a^2b và ab^2 .

CẢNH BÁO

Hàm `PolynomialFeatures(degree=d)` biến đổi một mảng chứa n đặc trưng thành một mảng chứa $(n + d)! / d!$ đặc trưng, trong đó $n!$ là giai thừa của n , bằng $1 \times 2 \times 3 \times \dots \times n$. Hãy cẩn thận với sự bùng nổ tổ hợp về số lượng đặc trưng!

Đường cong học tập

Nếu bạn thực hiện hồi quy đa thức bậc cao, bạn sẽ có được kết quả phù hợp với dữ liệu huấn luyện tốt hơn nhiều so với hồi quy tuyến tính thông thường. Ví dụ, [Hình 4-14](#) áp dụng mô hình đa thức bậc 300 cho dữ liệu huấn luyện ở trên và so sánh kết quả với mô hình tuyến tính thuần túy và mô hình bậc hai (đa thức bậc hai). Hãy chú ý cách mô hình đa thức bậc 300 dao động để càng gần với các trường hợp huấn luyện càng tốt.



Hình 4-14. Hồi quy đa thức bậc cao

Mô hình hồi quy đa thức bậc cao này đang bị quá khớp nghiêm trọng với dữ liệu huấn luyện, trong khi mô hình tuyến tính lại bị thiếu khớp. Mô hình có khả năng tổng quát hóa tốt nhất trong trường hợp này là mô hình bậc hai, điều này hợp lý vì dữ liệu được tạo ra bằng mô hình bậc hai. Nhưng nói chung, bạn sẽ không biết hàm nào đã tạo ra dữ liệu, vậy làm thế nào bạn có thể quyết định độ phức tạp của mô hình? Làm thế nào bạn có thể biết mô hình của mình đang bị quá khớp hay thiếu khớp với dữ liệu?

Trong [Chương 2](#), bạn đã sử dụng phương pháp kiểm định chéo để ước tính hiệu suất tổng quát hóa của mô hình. Nếu mô hình hoạt động tốt trên dữ liệu huấn luyện nhưng lại tổng quát hóa kém theo các chỉ số kiểm định chéo, thì mô hình đó bị quá khớp (overfitting). Nếu nó hoạt động kém trên cả hai, thì nó bị thiếu khớp (underfitting). Đây là một cách để biết khi nào một mô hình quá đơn giản hoặc quá phức tạp.

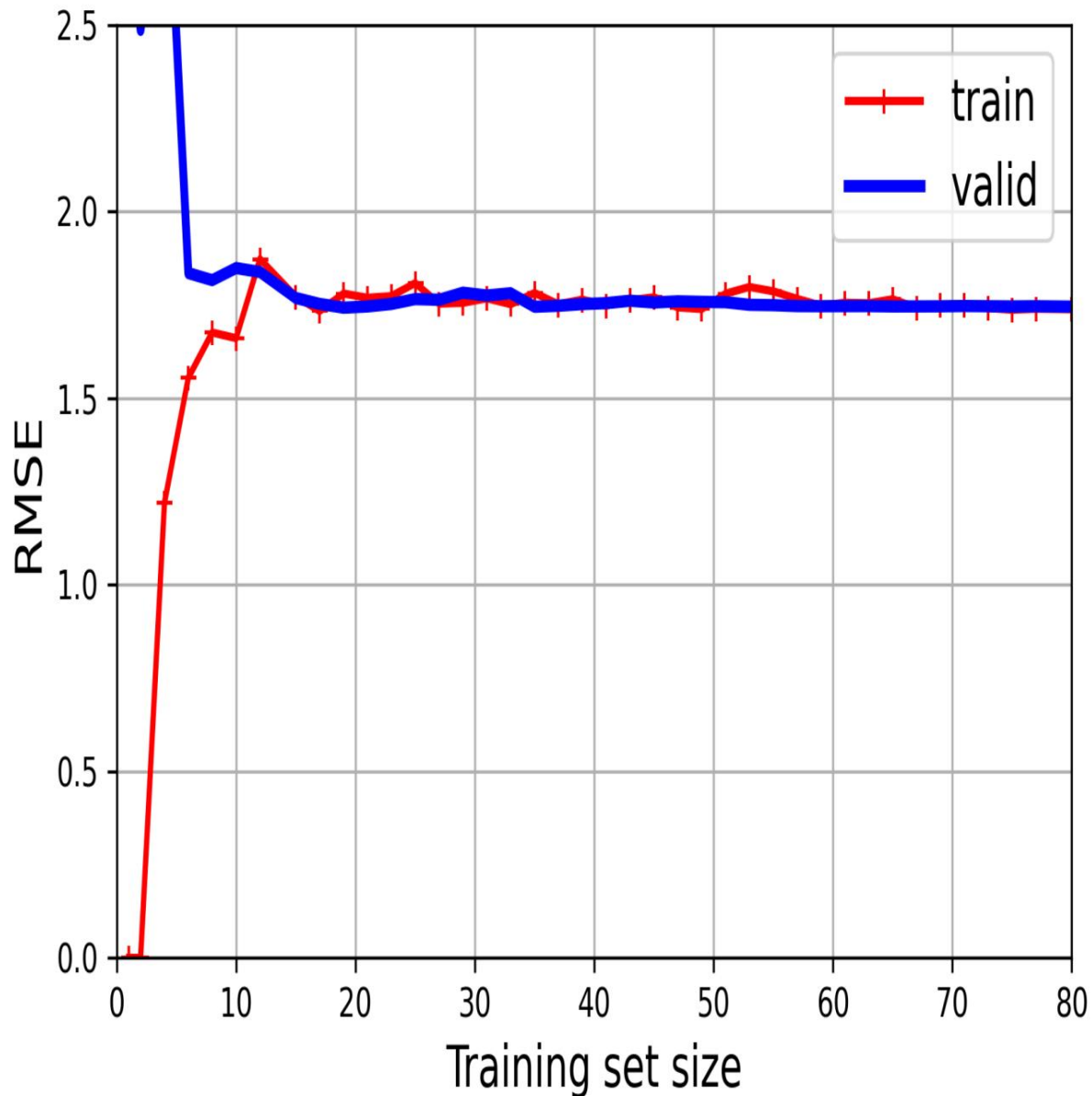
Một cách khác để nhận biết là xem xét đường cong học tập: đây là đồ thị biểu diễn lỗi huấn luyện và lỗi xác thực của mô hình theo số lần lặp huấn luyện: chỉ cần đánh giá mô hình theo định kỳ trong quá trình huấn luyện trên cả tập huấn luyện và tập xác thực, rồi vẽ đồ thị kết quả. Nếu mô hình không thể được huấn luyện tăng dần (tức là nếu nó không hỗ trợ `partial_fit()` hoặc `warm_start`), thì bạn phải huấn luyện mô hình nhiều lần trên các tập con lớn dần của tập huấn luyện.

Scikit-Learn có một hàm `learning_curve()` hữu ích để hỗ trợ việc này: nó huấn luyện và đánh giá mô hình bằng cách sử dụng phương pháp kiểm định chéo. Theo mặc định, nó huấn luyện lại mô hình trên các tập con ngày càng tăng của tập huấn luyện, nhưng nếu mô hình hỗ trợ học tăng dần, bạn có thể đặt `exploit_incremental_learning=True` khi gọi `learning_curve()` và nó sẽ huấn luyện mô hình một cách tăng dần. Hàm này trả về kích thước tập huấn luyện mà nó đã đánh giá mô hình, và điểm số huấn luyện và xác thực mà nó đo được cho mỗi kích thước và cho mỗi lần kiểm định chéo. Hãy sử dụng hàm này để xem đường cong học tập của mô hình Hồi quy tuyến tính đơn giản (xem [Hình 4-15](#)):

```
from sklearn.model_selection import learning_curve

train_sizes, train_scores, valid_scores =
    learning_curve( LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0,
        40), cv=5,
    scoring="neg_root_mean_squared_error")
train_errors = -train_scores.mean(axis=1) valid_errors = -valid_scores.mean(axis=1)

plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid")
[...] # Làm đẹp hình: thêm nhãn, trục, lưới và chú thích. plt.show()
```



Hình 4-15. Đường cong học tập

Mô hình này đang bị hiện tượng underfitting (không khớp dữ liệu), hãy cùng xem tại sao. Đầu tiên, hãy xem xét lỗi huấn luyện: khi chỉ có một hoặc hai trường hợp trong tập huấn luyện, mô hình có thể khớp chúng một cách hoàn hảo, đó là lý do tại sao đường cong bắt đầu từ số 0. Nhưng khi các trường hợp mới được thêm vào tập huấn luyện, mô hình không thể khớp dữ liệu huấn luyện một cách hoàn hảo, cả vì dữ liệu bị nhiễu và vì nó hoàn toàn không tuyến tính. Vì vậy, lỗi trên dữ liệu huấn luyện tăng lên cho đến khi đạt đến mức ổn định, tại thời điểm đó việc thêm các trường hợp mới vào tập huấn luyện không làm cho lỗi trung bình tốt hơn hoặc tệ hơn nhiều. Bây giờ hãy xem xét lỗi xác thực. Khi mô hình được huấn luyện trên rất ít trường hợp huấn luyện, nó không có khả năng khái quát hóa đúng cách, đó là lý do tại sao lỗi xác thực ban đầu khá lớn. Sau đó, khi mô hình được cung cấp nhiều ví dụ huấn luyện hơn, nó học hỏi, và do đó lỗi xác thực giảm dần. Tuy nhiên, một lần nữa, một đường thẳng không thể mô hình hóa dữ liệu một cách tốt, vì vậy lỗi cuối cùng đạt đến mức ổn định, rất gần với đường cong kia.

Các đường cong học tập này là điển hình của một mô hình chưa khớp (underfitting). Cả hai đường cong đều đã đạt đến điểm bão hòa; chúng gần nhau và khá cao.

MẸO

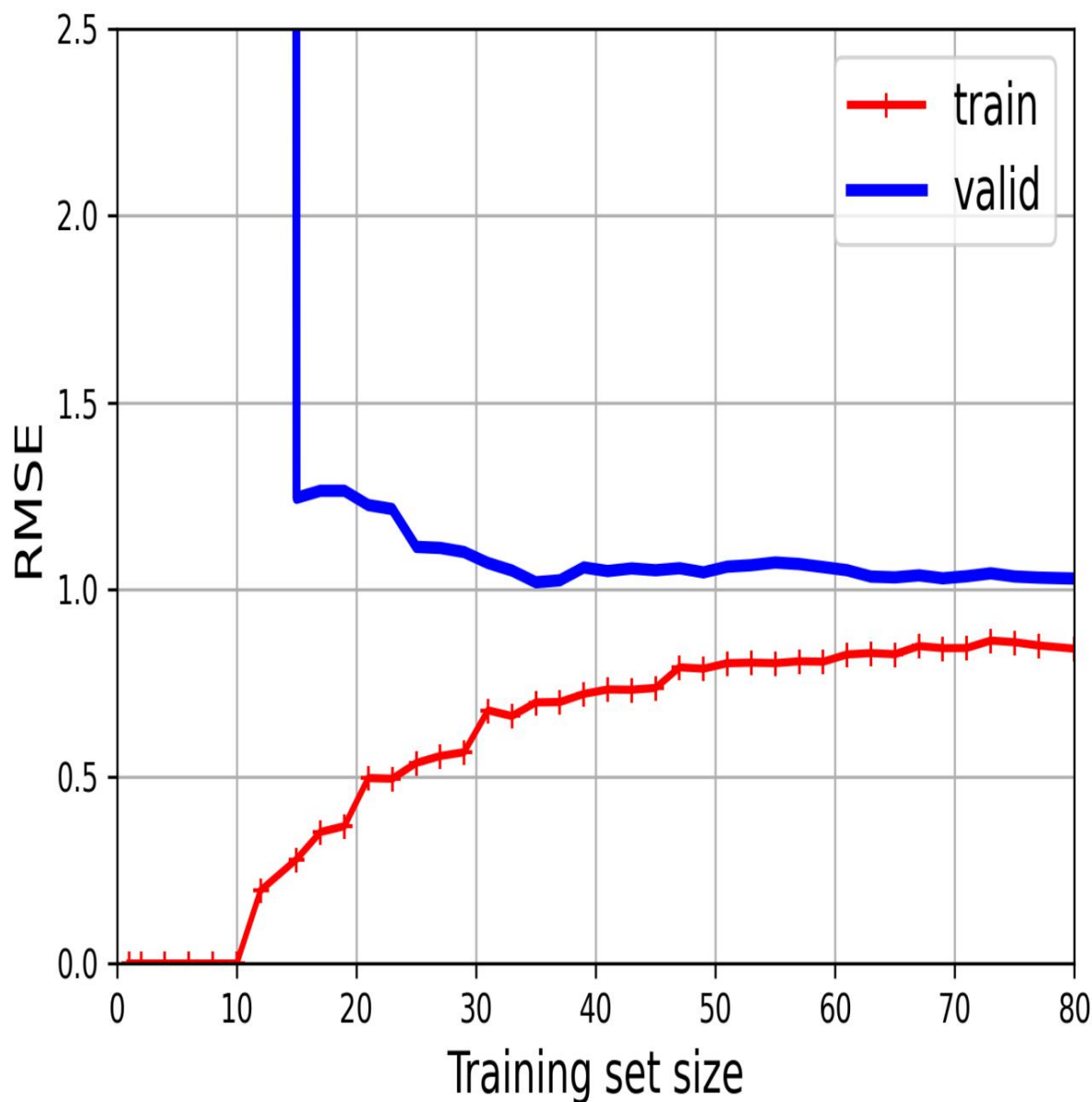
Nếu mô hình của bạn không khớp với dữ liệu huấn luyện, việc thêm nhiều ví dụ huấn luyện sẽ không giúp ích gì. Bạn cần sử dụng một mô hình tốt hơn hoặc tìm ra các đặc trưng tốt hơn.

Bây giờ chúng ta hãy xem xét đường cong học tập của mô hình đa thức bậc 10 trên cùng dữ liệu (Hình 4-16):

```
from sklearn.pipeline import make_pipeline

hồi quy đa thức = tạo đường ống (
    PolynomialFeatures(degree=10, include_bias=False),
    Hồi quy tuyến tính())

kích thước huấn luyện, điểm huấn luyện, điểm hợp lệ = đường cong học tập(
    polynomial_regression, X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error") [...] # giống
như trước đó
```



Hình 4-16. Đường cong học tập cho mô hình đa thức bậc 10

Quá trình học tập này trông có vẻ tương tự như những quá trình trước đó, nhưng có hai điểm khác biệt rất quan trọng:

- Sai số trên dữ liệu huấn luyện thấp hơn nhiều so với trước đây.
- Có một khoảng cách giữa hai đường cong. Điều này có nghĩa là mô hình hoạt động tốt hơn đáng kể trên dữ liệu huấn luyện so với dữ liệu kiểm định, đây là dấu hiệu đặc trưng của một mô hình bị quá khớp (overfitting). Tuy nhiên, nếu bạn sử dụng một tập dữ liệu huấn luyện lớn hơn nhiều, hai đường cong sẽ tiếp tục tiến gần nhau hơn.

MẸO

Một cách để cải thiện mô hình bị quá khớp là cung cấp thêm dữ liệu huấn luyện cho đến khi lỗi trên tập dữ liệu kiểm chứng đạt đến lỗi trên tập dữ liệu huấn luyện.

SỰ ĐÁNH ĐỔI GIỮA THIÊN VỊ VÀ PHƯƠNG SAI

Một kết quả lý thuyết quan trọng của thống kê và máy học là thực tế rằng sai số tổng quát của một mô hình có thể được biểu thị bằng tổng của ba sai số rất khác nhau:

Thiên kiến

Lỗi khái quát hóa phần này là do những giả định sai lầm, chẳng hạn như giả định rằng dữ liệu là tuyến tính trong khi thực tế là bậc hai. Một mô hình có độ lệch cao rất dễ bị thiếu khớp với dữ liệu huấn luyện.

Phương sai

Nguyên nhân là do mô hình quá nhạy cảm với những biến đổi nhỏ trong dữ liệu huấn luyện. Một mô hình có nhiều bậc tự do (như mô hình đa thức bậc cao) thường có phương sai cao và do đó dẫn đến hiện tượng quá khớp dữ liệu huấn luyện.

Lỗi không thể khắc phục

Nguyên nhân của lỗi này là do độ nhiễu của chính dữ liệu. Cách duy nhất để giảm thiểu lỗi này là làm sạch dữ liệu (ví dụ: sửa chữa các nguồn dữ liệu, chẳng hạn như cảm biến bị hỏng, hoặc phát hiện và loại bỏ các giá trị ngoại lệ).

Việc tăng độ phức tạp của mô hình thường sẽ làm tăng phương sai và giảm độ lệch. Ngược lại, việc giảm độ phức tạp của mô hình sẽ làm tăng độ lệch và giảm phương sai. Đó là lý do tại sao nó được gọi là sự đánh đổi.

6

Mô hình tuyến tính được điều chỉnh

Như chúng ta đã thấy trong Chương 1 và 2, một cách tốt để giảm thiểu hiện tượng quá khớp (overfitting) là điều chỉnh mô hình (tức là ràng buộc nó): càng ít bậc tự do thì mô hình càng khó bị quá khớp với dữ liệu. Một cách đơn giản để điều chỉnh mô hình đa thức là giảm số bậc của đa thức.

Đối với mô hình tuyến tính, việc điều chỉnh thường được thực hiện bằng cách ràng buộc các trọng số của mô hình. Chúng ta sẽ xem xét Hồi quy Ridge, Hồi quy Lasso và Elastic Net, ba phương pháp khác nhau để ràng buộc các trọng số.

Hồi quy Ridge

Hồi quy Ridge (còn gọi là chuẩn hóa Tikhonov) là một phiên bản được chuẩn hóa của hồi quy tuyến tính: một hệ số chuẩn hóa bằng α nhân dữ liệu nhưng đồng thời giữ cho $\sum_{i=1}^n \theta_i^2$ được thêm vào MSE. Điều này buộc thuật toán học không chỉ phải phù hợp của mô hình càng nhỏ càng tốt. Lưu ý rằng hệ số chuẩn hóa chỉ nên được thêm vào hàm chi phí trong quá trình huấn luyện. Sau khi mô hình được huấn luyện, bạn muốn sử dụng MSE không được chuẩn hóa (hoặc RMSE) để đánh giá hiệu suất của mô hình.

Siêu tham số α kiểm soát mức độ bạn muốn điều chỉnh mô hình. Nếu $\alpha = 0$, thì Hồi quy Ridge chỉ đơn giản là Hồi quy tuyến tính. Nếu α rất lớn, thì tất cả các trọng số sẽ rất gần bằng 0 và kết quả là một đường thẳng nằm ngang đi qua giá trị trung bình của dữ liệu. Phương trình 4-8 trình bày hàm chi phí của Hồi quy Ridge.

Phương trình 4-8. Hàm chi phí hồi quy Ridge

$$J(\mathbf{w}) = \text{MSE}(\mathbf{w}) + \alpha \sum_{i=1}^n \theta_i^2$$

Lưu ý rằng số hạng độ lệch θ không được điều chỉnh (tổng bắt đầu từ $i = 1$, chứ không phải 0). Nếu ta định nghĩa $\mathbf{w}$ là vectơ gồm θ trọng số đặc trưng (θ đến θ), thì số hạng điều chỉnh bằng $\alpha(\|\mathbf{w}\|_2)^2 / 2$, trong đó $\|\mathbf{w}\|_2$ biểu thị ℓ_2 chuẩn của vectơ trọng số. Đối với Batch Gradient Descent, chỉ cần thêm $2\alpha \mathbf{w} / m$ vào phần gradient MSE.

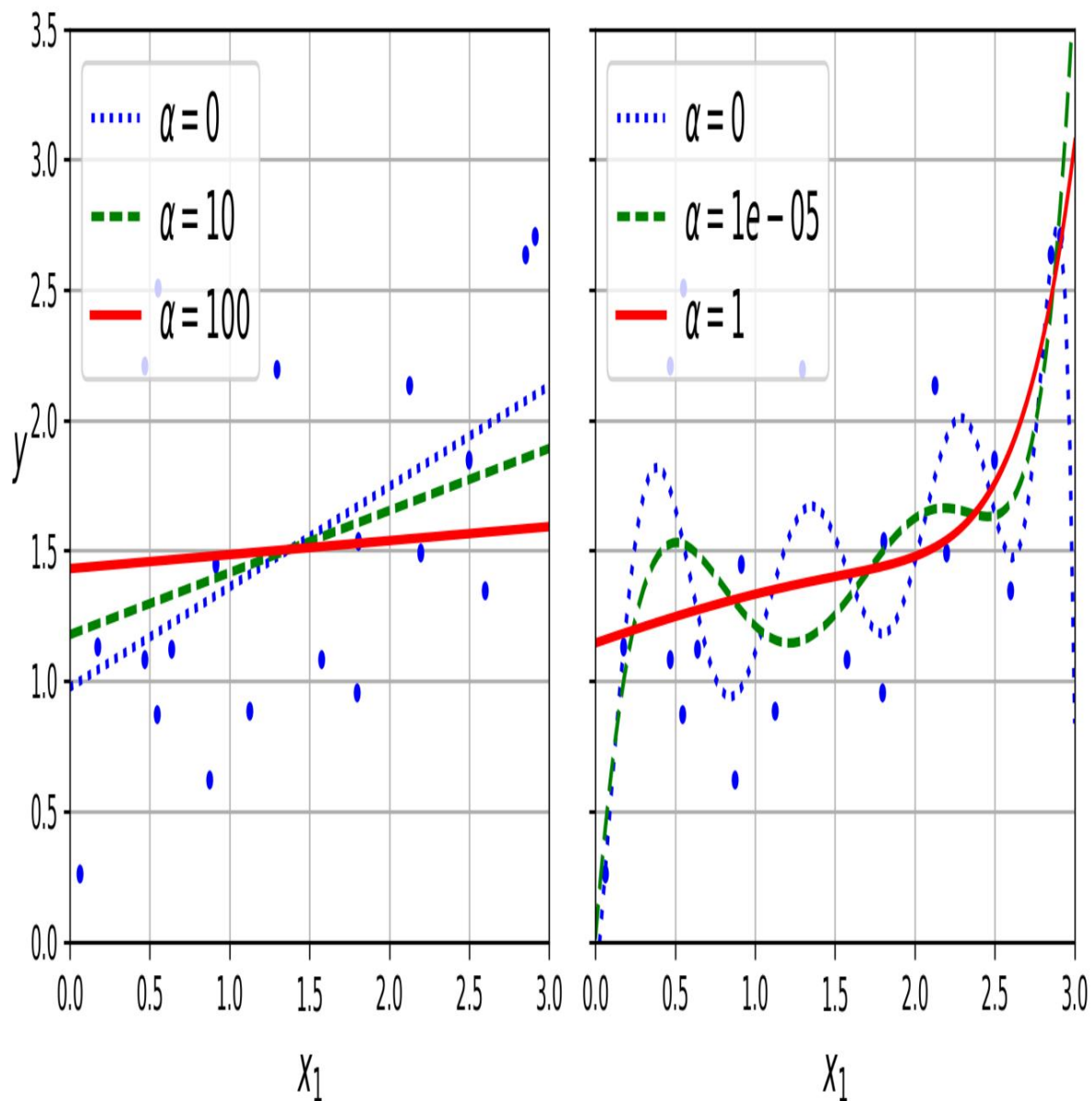
2 2 2 2 8

vectơ tương ứng với trọng số đặc trưng, mà không cộng thêm bất cứ điều gì vào độ dốc của số hạng thiên lệch (xem [Phương trình 4-6](#)).

CẢNH BÁO

Việc chuẩn hóa dữ liệu (ví dụ: sử dụng `StandardScaler`) trước khi thực hiện hồi quy Ridge là rất quan trọng, vì phương pháp này nhạy cảm với độ phân giải của các đặc trưng đầu vào. Điều này đúng với hầu hết các mô hình được điều chỉnh.

Hình 4-17 minh họa một số mô hình Ridge được huấn luyện trên dữ liệu tuyến tính rất nhiễu bằng cách sử dụng các giá trị α khác nhau. Bên trái, các mô hình Ridge thông thường được sử dụng, dẫn đến các dự đoán tuyến tính. Bên phải, dữ liệu được mở rộng trước bằng cách sử dụng `PolynomialFeatures(degree=10)`, sau đó được chuẩn hóa bằng `StandardScaler`, và cuối cùng các mô hình Ridge được áp dụng cho các đặc trưng thu được: đây là Hồi quy Đa thức với chuẩn hóa Ridge. Lưu ý rằng việc tăng α dẫn đến các dự đoán phẳng hơn (tức là ít cực đoan hơn, hợp lý hơn), do đó làm giảm phương sai của mô hình nhưng làm tăng độ lệch của nó.



Hình 4-17. Mô hình tuyến tính (bên trái) và mô hình đa thức (bên phải), cả hai đều có các mức độ điều chỉnh Ridge khác nhau.

Cũng như hồi quy tuyến tính, chúng ta có thể thực hiện hồi quy Ridge bằng cách tính toán phương trình dạng đóng hoặc bằng cách thực hiện thuật toán giảm độ dốc. Ưu điểm và nhược điểm của cả hai phương pháp đều giống nhau. Phương trình 4-9 thể hiện nghiệm dạng đóng, trong đó A là ma trận đơn vị $(n + 1) \times (n + 1)$, ngoại trừ ô trên cùng bên trái có giá trị 0, tương ứng với hệ số thiên vị.

thuật ngữ.

Phương trình 4-9. Giải pháp dạng đóng của hồi quy Ridge

$$\hat{\beta} = (X^T X + \alpha A)^{-1} X^T y$$

Dưới đây là cách thực hiện hồi quy Ridge với Scikit-Learn bằng cách sử dụng giải pháp dạng đóng (một biến thể của Phương trình 4-9 sử dụng kỹ thuật phân tích ma trận của André-Louis Cholesky):

```
>>> from sklearn.linear_model import Ridge >>> ridge_reg =
Ridge(alpha=0.1, solver="cholesky") >>> ridge_reg.fit(X, y)
```

```
>>> ridge_reg.predict([[1.5]])
array([[1.55325833]])
```

Và sử dụng thuật toán tối ưu hóa độ dốc ngẫu nhiên (Stochastic Gradient Descent): **10**

```
>>> sgd_reg = SGDRegressor(penalty="l2", alpha=0.1 / m, tol=None,
...                        max_iter=1000, eta0=0.01, random_state=42)
...
>>> sgd_reg.fit(X, y.ravel()) # y.ravel() vì fit() mong đợi mục tiêu 1D >>>
sgd_reg.predict([[1.5]])
array([1.55302613])
```

Siêu tham số phạt xác định loại số hạng điều chỉnh cần sử dụng. Việc chỉ định "l2" cho biết bạn muốn SGD thêm một số hạng điều chỉnh vào hàm chi phí MSE bằng alpha nhân với bình phương của chuẩn l của vectơ trọng số: điều này giống như Hồi quy Ridge, ngoại trừ 2 trường hợp này không có phép chia cho m, đó là lý do tại sao chúng ta truyền alpha=0.1 / m, để có được kết quả tương tự như Ridge(alpha=0.1).

MẸO

Lớp RidgeCV cũng thực hiện hồi quy Ridge, nhưng nó tự động điều chỉnh các siêu tham số bằng cách sử dụng kiểm định chéo. Về cơ bản, nó tương đương với việc sử dụng GridSearchCV, nhưng được tối ưu hóa cho hồi quy Ridge và chạy nhanh hơn nhiều. Một số thuật toán ước lượng khác (chủ yếu là tuyến tính) cũng có các biến thể kiểm định chéo hiệu quả, chẳng hạn như LassoCV hoặc ElasticNetCV.

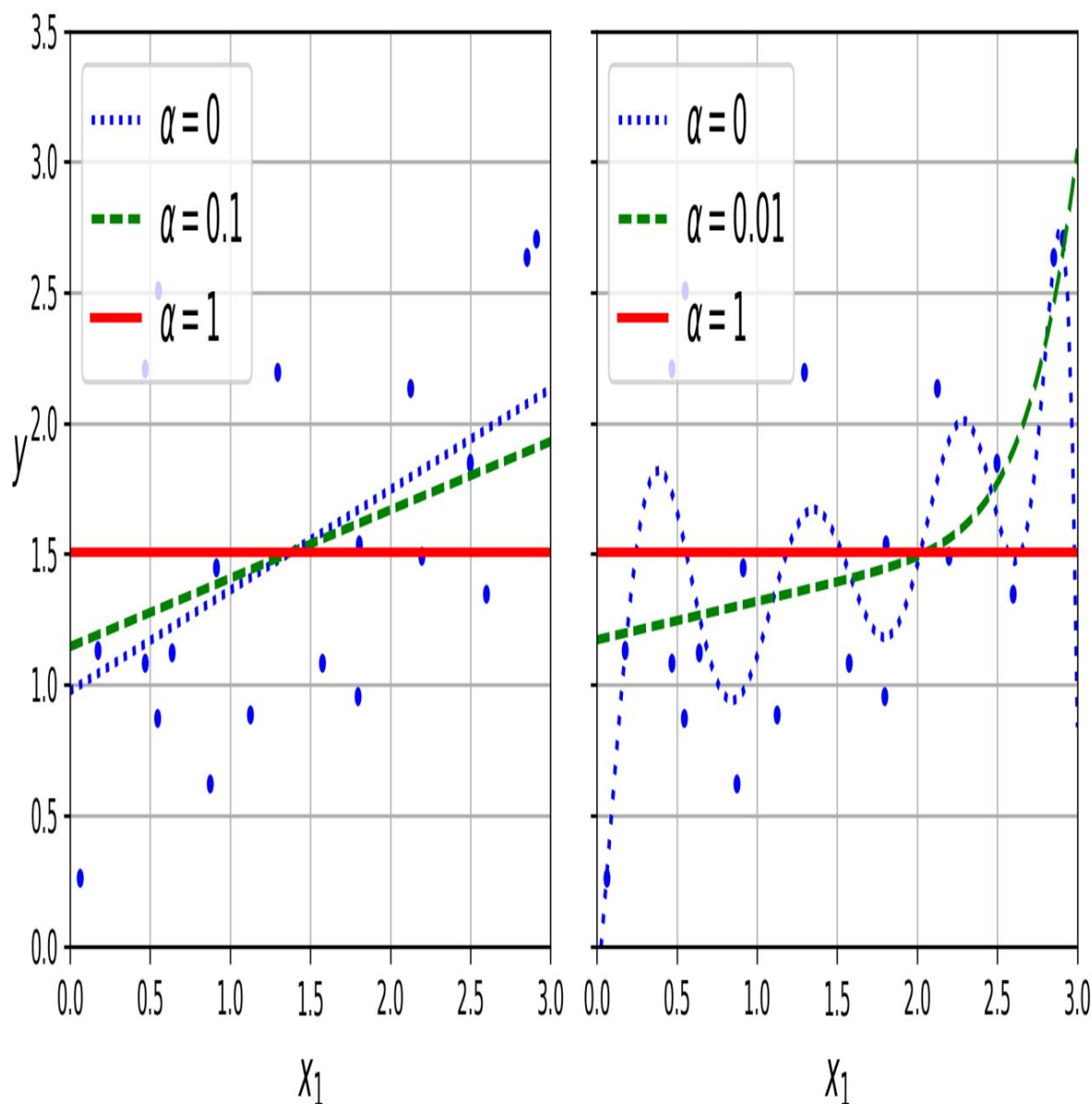
Lasso Regression

Hồi quy Least Absolute Shrinkage and Selection Operator (thường được gọi đơn giản là Hồi quy Lasso) là một phiên bản được điều chỉnh khác của Hồi quy tuyến tính: giống như Hồi quy Ridge, nó thêm một thuật ngữ điều chỉnh vào hàm chi phí, nhưng nó sử dụng chuẩn l của vectơ trọng số thay vì bình phương của chuẩn l (xem **Phương trình 4-10**). Lưu ý rằng chuẩn l được nhân với 2_α, trong khi chuẩn l được nhân với α / m trong hồi quy Ridge. Các hệ số này được chọn để đảm bảo rằng giá trị α tối ưu không phụ thuộc vào kích thước tập huấn luyện: các chuẩn khác nhau dẫn đến các hệ số khác nhau (xem vấn đề #15657 của Scikit-Learn để biết thêm chi tiết).

Phương trình 4-10. Hàm chi phí hồi quy Lasso

$$J(\beta) = \text{MSE}(\beta) + 2\alpha \sum_{i=1}^p |\beta_i|$$

Hình 4-18 thể hiện điều tương tự như **Hình 4-17** nhưng thay thế mô hình Ridge bằng mô hình Lasso và sử dụng các giá trị α khác nhau.

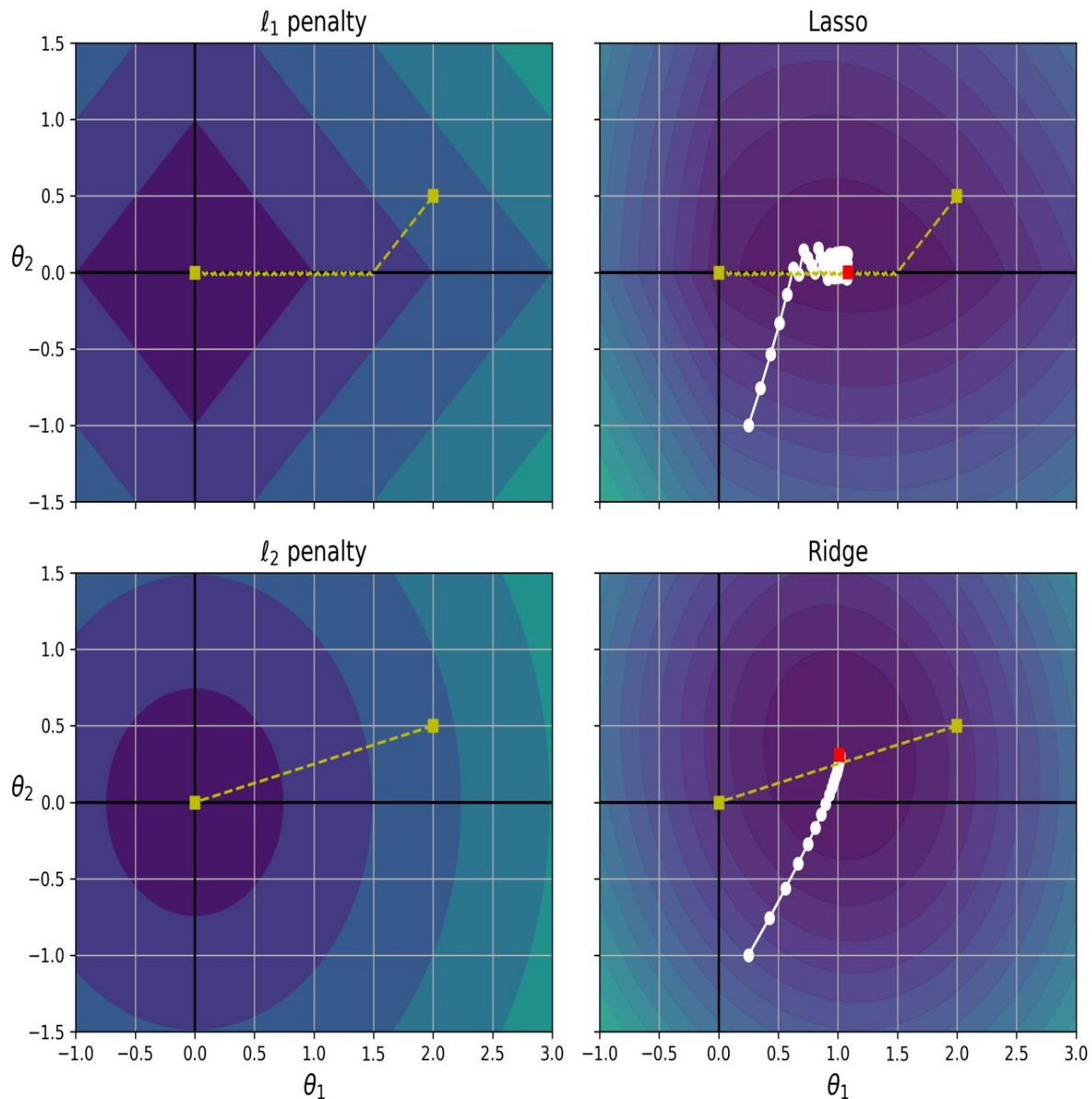


Hình 4-18. Mô hình tuyến tính (bên trái) và mô hình đa thức (bên phải), cả hai đều sử dụng các mức độ điều chỉnh Lasso khác nhau.

Một đặc điểm quan trọng của Lasso Regression là nó có xu hướng loại bỏ trọng số của các đặc trưng ít quan trọng nhất (tức là đặt chúng bằng không). Ví dụ, đường nét đứt trong biểu đồ bên phải ở Hình 4-18 (với $\alpha = 0,01$) trông gần giống như hàm bậc ba: tất cả các trọng số cho các đặc trưng đa thức bậc cao đều bằng không. Nói cách khác, Lasso Regression tự động thực hiện chọn lọc đặc trưng và xuất ra một mô hình thưa (tức là, với ít trọng số đặc trưng khác không).

Bạn có thể hiểu lý do tại sao lại như vậy bằng cách xem Hình 4-19: các trục biểu thị hai tham số mô hình, và các đường đồng mức nền biểu thị các hàm mất mát khác nhau. Trong biểu đồ phía trên bên trái, các đường đồng mức biểu thị hàm mất mát ℓ_2 ($\| \theta \|^2$), giảm tuyến tính khi bạn tiến gần đến bất kỳ trục nào. Ví dụ, nếu bạn khởi tạo các tham số mô hình là $\theta = 2$ và $\phi = 0,5$, việc chạy thuật toán Gradient Descent sẽ giảm cả hai tham số một cách đồng đều (như được biểu thị bằng đường màu vàng đứt nét); do đó θ sẽ đạt đến 0 trước (vì ban đầu nó gần 0 hơn). Sau đó, thuật toán Gradient Descent sẽ lăn xuống rãnh cho đến khi đạt đến $\theta = 0$ (với một chút dao động, vì độ dốc của ℓ không bao giờ tiến gần đến 0: chúng luôn bằng -1 hoặc 1 cho mỗi tham số). Trong biểu đồ phía trên bên phải, các đường đồng mức biểu thị hàm chi phí của Lasso (tức là hàm chi phí MSE cộng với tổng nhất ℓ_1). Các vòng tròn nhỏ màu trắng.

Hãy xem đường đi mà thuật toán Gradient Descent thực hiện để tối ưu hóa một số tham số mô hình được khởi tạo xung quanh $\theta = 0.25$ và $\theta = -1$: hãy chú ý một lần nữa cách đường đi nhanh chóng đạt đến $\theta = 0$, sau đó lăn xuống rãnh và cuối cùng dao động quanh điểm tối ưu toàn cục (được biểu thị bằng hình vuông màu đỏ). Nếu chúng ta tăng α , điểm tối ưu toàn cục sẽ di chuyển sang trái dọc theo đường chấm vàng, trong khi nếu chúng ta giảm α , điểm tối ưu toàn cục sẽ di chuyển sang phải (trong ví dụ này, các tham số tối ưu cho MSE không được điều chỉnh là $\theta \approx 2$ và $\theta = 0.5$).



Hình 4-19. So sánh Lasso và phương pháp điều chỉnh Ridge.

Hai đồ thị phía dưới thể hiện cùng một điều nhưng với hình phạt ℓ thay vì α . Trong đồ thị phía dưới bên trái, bạn có thể thấy rằng tổn thất ℓ giảm khi chúng ta tiến gần đến gốc tọa độ, vì vậy Gradient Descent chỉ đi theo một đường thẳng đến điểm đó. Trong đồ thị phía dưới bên phải, các đường đồng mức biểu thị hàm chi phí của Ridge Regression (tức là hàm chi phí MSE cộng với tổn thất ℓ). Như bạn có thể thấy, độ dốc giảm dần khi các tham số tiến gần đến điểm tối ưu toàn cục, vì vậy Gradient Descent tự nhiên chậm lại, điều này giúp hội tụ (vì không có sự dao động). Điều này giúp Ridge hội tụ nhanh hơn Lasso. Cũng cần lưu ý rằng các tham số tối ưu (được biểu thị bằng hình vuông màu đỏ) ngày càng tiến gần đến gốc tọa độ khi bạn tăng α , nhưng chúng không bao giờ bị loại bỏ hoàn toàn.

MẸO

Để tránh hiện tượng Gradient Descent dao động quanh điểm tối ưu ở cuối quá trình khi sử dụng Lasso, bạn cần giảm dần tốc độ học trong quá trình huấn luyện (nó vẫn sẽ dao động quanh điểm tối ưu, nhưng các bước sẽ nhỏ dần, do đó nó sẽ hội tụ).

Hàm chi phí Lasso không khả vi tại $\theta = 0$ (với $i = 1, 2, \dots, n$), nhưng thuật toán Gradient Descent vẫn hoạt động nếu bạn sử dụng vectơ đạo hàm riêng g thay thế khi $\theta = 0$. Phương trình 4-11 thể hiện phương trình vectơ đạo hàm riêng mà bạn có thể sử dụng cho thuật toán Gradient Descent với hàm chi phí Lasso.

Phương trình 4-11. Vectơ đạo hàm riêng của hồi quy Lasso

$$g(\theta) = \begin{bmatrix} \text{dấu}(\theta_1) \\ \text{dấu}(\theta_2) \\ \vdots \\ \text{dấu}(\theta_n) \end{bmatrix} + 2\alpha \begin{bmatrix} 1 \text{ nếu } \theta_i < 0 \\ 0 \text{ nếu } \theta_i = 0 \\ -1 \text{ nếu } \theta_i > 0 \end{bmatrix}$$

trong đó $\text{dấu}(\theta_i) = \begin{cases} 1 & \text{nếu } \theta_i < 0 \\ 0 & \text{nếu } \theta_i = 0 \\ -1 & \text{nếu } \theta_i > 0 \end{cases}$

Dưới đây là một ví dụ nhỏ sử dụng lớp Lasso trong Scikit-Learn:

```
>>> from sklearn.linear_model import Lasso
>>> lasso_reg = Lasso(alpha=0.1)
>>> lasso_reg.fit(X, y)
>>> lasso_reg.predict([[1.5]])
array([1.53788174])
```

Lưu ý rằng bạn cũng có thể sử dụng `SGDRegressor(penalty="l1", alpha=0.1)`.

Lưới đàn hồi

Elastic Net là một phương pháp trung gian giữa Ridge Regression và Lasso Regression. Hệ số điều chỉnh là tổng có trọng số của cả hệ số điều chỉnh của Ridge và Lasso, và bạn có thể kiểm soát tỷ lệ pha trộn r . Khi $r = 0$, Elastic Net tương đương với Ridge Regression, và khi $r = 1$, nó tương đương với Lasso Regression (xem Phương trình 4-12).

Phương trình 4-12. Hàm chi phí lưới đàn hồi

$$J(\theta) = \text{MSE}(\theta) + r(2\alpha \sum_{i=1}^n |\theta_i|) + (1 - r)(\alpha \sum_{i=1}^n \theta_i^2)$$

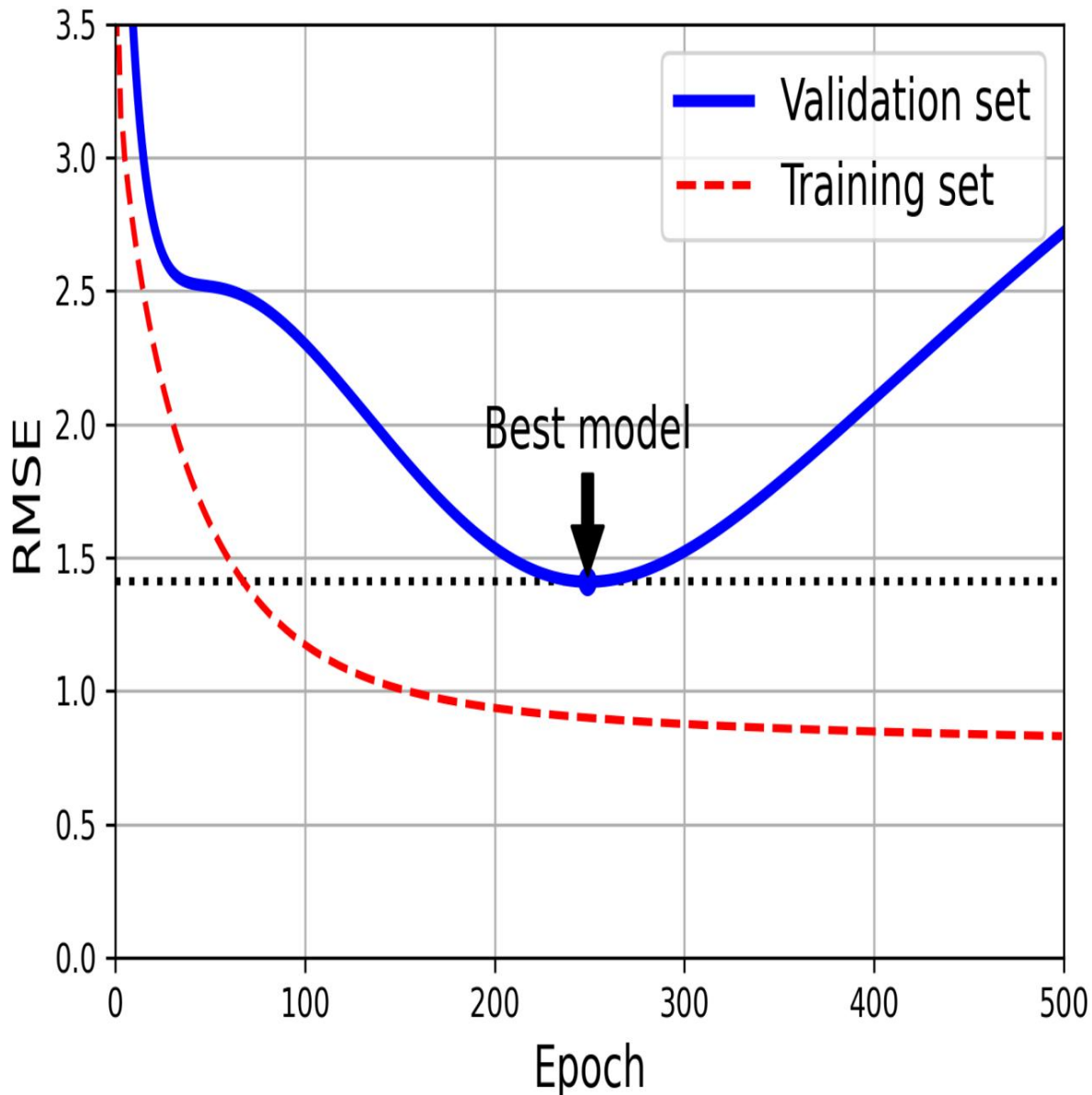
Vậy khi nào bạn nên sử dụng Hồi quy tuyến tính thông thường (tức là không có bất kỳ chuẩn hóa nào), Ridge, Lasso hay Elastic Net? Hầu như luôn luôn nên có ít nhất một chút chuẩn hóa, vì vậy nói chung bạn nên tránh Hồi quy tuyến tính thông thường. Ridge là một lựa chọn mặc định tốt, nhưng nếu bạn nghi ngờ rằng chỉ có một vài đặc trưng hữu ích, bạn nên ưu tiên Lasso hoặc Elastic Net vì chúng có xu hướng giảm trọng số của các đặc trưng không hữu ích xuống bằng không, như chúng ta đã thảo luận. Nói chung, Elastic Net được ưu tiên hơn Lasso vì Lasso có thể hoạt động thất thường khi số lượng đặc trưng lớn hơn số lượng mẫu huấn luyện hoặc khi một số đặc trưng có tương quan mạnh.

Dưới đây là một ví dụ ngắn sử dụng ElasticNet của Scikit-Learn (`l1_ratio` tương ứng với tỷ lệ pha trộn r):

```
>>> from sklearn.linear_model import ElasticNet
>>> elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
>>> elastic_net.fit(X, y)
>>> elastic_net.predict([[1.5]])
array([1.54333232])
```

Dừng sớm

Một cách rất khác để điều chỉnh các thuật toán học lặp như Gradient Descent là dừng huấn luyện ngay khi lỗi xác thực đạt mức tối thiểu. Điều này được gọi là dừng sớm (early stopping). Hình 4-20 cho thấy một mô hình phức tạp (trong trường hợp này, là mô hình hồi quy đa thức bậc cao) được huấn luyện bằng Batch Gradient Descent trên tập dữ liệu bậc hai mà chúng ta đã sử dụng trước đó. Khi số epoch trôi qua, thuật toán học hỏi và lỗi dự đoán (RMSE) trên tập huấn luyện giảm xuống, cùng với lỗi dự đoán trên tập xác thực. Tuy nhiên, sau một thời gian, lỗi xác thực ngừng giảm và bắt đầu tăng trở lại. Điều này cho thấy mô hình đã bắt đầu quá khớp với dữ liệu huấn luyện. Với dừng sớm, bạn chỉ cần dừng huấn luyện ngay khi lỗi xác thực đạt mức tối thiểu. Đó là một kỹ thuật điều chỉnh đơn giản và hiệu quả đến mức Geoffrey Hinton gọi nó là "một bữa trưa miễn phí tuyệt vời".



Hình 4-20. Điều chỉnh dừng sớm

MẸO

Với thuật toán Gradient Descent ngẫu nhiên và theo lô nhỏ, các đường cong không được mượt mà lắm, và có thể khó biết liệu bạn đã đạt đến điểm cực tiểu hay chưa. Một giải pháp là chỉ dừng lại sau khi lỗi xác thực đã cao hơn điểm cực tiểu trong một khoảng thời gian nhất định (khi bạn tự tin rằng mô hình sẽ không hoạt động tốt hơn nữa), sau đó điều chỉnh lại các tham số của mô hình về điểm mà lỗi xác thực ở mức tối thiểu.

Dưới đây là một ví dụ cơ bản về việc dừng sớm:

```
from copy import deepcopy from
sklearn.metrics import mean_squared_error from
sklearn.preprocessing import StandardScaler

X_train, y_train, X_valid, y_valid = [...] # chia tập dữ liệu bậc hai

tiền xử lý = tạo đường ống (Đặc trưng đa thức (bậc=90, bao gồm độ lệch=False),
                             StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid) sgd_reg =
SGDRegressor(penalty=None, eta0=0.002, random_state=42) n_epochs = 500 best_valid_rmse
= float('inf')

for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep) val_error =
    mean_squared_error(y_valid, y_valid_predict, squared=False) if val_error < best_valid_rmse:
        best_valid_rmse = val_error best_model =
        deepcopy(sgd_reg)
```

Đoạn mã này thực hiện như sau: đầu tiên, nó thêm các đặc trưng đa thức và chuẩn hóa tất cả các đặc trưng đầu vào, cả cho tập huấn luyện và tập kiểm định (đoạn mã này giả định rằng bạn đã chia tập huấn luyện thành một tập huấn luyện nhỏ hơn và một tập kiểm định). Sau đó, nó tạo một mô hình SGDRegressor không có chuẩn hóa và tốc độ học nhỏ. Trong vòng lặp huấn luyện, nó gọi `partial_fit()` thay vì `fit()` để thực hiện học tăng dần. Ở mỗi epoch, nó đo RMSE trên tập kiểm định. Nếu RMSE thấp hơn RMSE thấp nhất đã thấy cho đến nay, nó sẽ lưu một bản sao của mô hình vào biến `best_model`. Việc triển khai này thực chất không dừng quá trình huấn luyện, nhưng nó cho phép bạn quay lại mô hình tốt nhất sau khi huấn luyện. Lưu ý rằng mô hình được sao chép bằng `copy.deepcopy()`, vì nó sao chép cả siêu tham số của mô hình và các tham số đã học. Ngược lại, `sklearn.base.clone()` chỉ sao chép siêu tham số của mô hình.

Hồi quy Logistic

Như chúng ta đã thảo luận trong [Chương 1](#), một số thuật toán hồi quy có thể được sử dụng cho phân loại (và ngược lại). Hồi quy Logistic (còn gọi là hồi quy Logit) thường được sử dụng để ước tính xác suất một trường hợp thuộc về một lớp cụ thể (ví dụ: xác suất email này là thư rác là bao nhiêu?). Nếu xác suất ước tính lớn hơn một ngưỡng nhất định (thường là 50%), thì mô hình dự đoán rằng trường hợp đó thuộc về lớp đó (gọi là lớp tích cực, được gán nhãn “1”), và ngược lại, nó dự đoán rằng trường hợp đó không thuộc về lớp đó (tức là nó thuộc về lớp tiêu cực, được gán nhãn “0”). Điều này làm cho nó trở thành một bộ phân loại nhị phân.

Ước tính xác suất

Vậy mô hình hồi quy logistic hoạt động như thế nào? Giống như mô hình hồi quy tuyến tính, mô hình hồi quy logistic tính tổng có trọng số của các đặc trưng đầu vào (cộng thêm một số hạng sai lệch), nhưng thay vì xuất ra kết quả trực tiếp như mô hình hồi quy tuyến tính, nó xuất ra giá trị logistic của kết quả này (xem [Phương trình 4-13](#)).

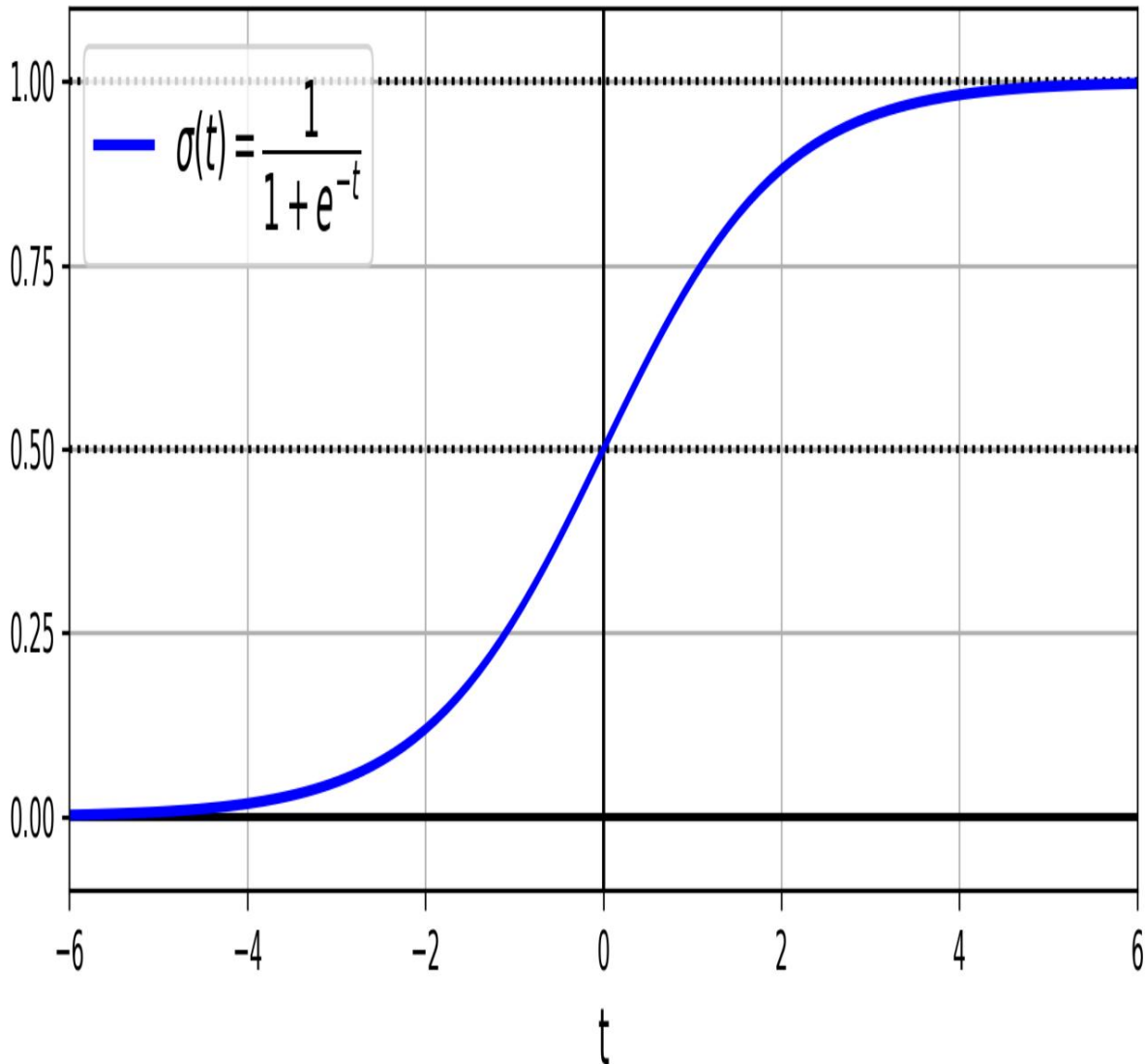
Phương trình 4-13. Mô hình hồi quy logistic ước tính xác suất (dạng vectơ)

$$\hat{p} = h(x) = \sigma(x)$$

Hàm logistic—ký hiệu là $\sigma(\cdot)$ —là một hàm sigmoid (tức là có dạng chữ S) cho ra một số nằm giữa 0 và 1. Nó được định nghĩa như trong [Phương trình 4-14](#) và [Hình 4-21](#).

Phương trình 4-14. Hàm logistic

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$



Hình 4-21. Hàm logistic

Khi mô hình Hồi quy Logistic đã ước tính xác suất $\hat{p} = h(x)$ rằng một trường hợp x thuộc về lớp tích cực, nó có thể dễ dàng đưa ra dự đoán $\hat{y}$ của mình (xem [Phương trình 4-15](#)).

Phương trình 4-15. Dự đoán mô hình hồi quy logistic sử dụng xác suất ngưỡng 50%.

$$0 \text{ nếu } \hat{p} < 0,5$$
$$\hat{y} = \begin{cases} 1 & \text{nếu } \hat{p} \geq 0,5 \end{cases}$$

Lưu ý rằng $\sigma(t) < 0,5$ khi $t < 0$ và $\sigma(t) \geq 0,5$ khi $t \geq 0$, do đó mô hình Hồi quy Logistic sử dụng ngưỡng xác suất mặc định là 50% dự đoán 1 nếu $\theta \cdot x$ dương và 0 nếu nó âm.

GHI CHÚ

Điểm số t thường được gọi là logit. Tên gọi này xuất phát từ thực tế rằng hàm logit, được định nghĩa là $\text{logit}(p) = \log(p / (1 - p))$, là hàm nghịch đảo của hàm logistic. Thật vậy, nếu bạn tính logit của xác suất ước tính p , bạn sẽ thấy kết quả là t . Logit cũng được gọi là log-odds, vì nó là logarit của tỷ lệ giữa xác suất ước tính cho lớp tích cực và xác suất ước tính cho lớp tiêu cực.

Chức năng đào tạo và chi phí

Giờ bạn đã biết cách mô hình Hồi quy Logistic ước tính xác suất và đưa ra dự đoán. Nhưng nó được huấn luyện như thế nào? Mục tiêu của việc huấn luyện là thiết lập vectơ tham số θ sao cho mô hình ước tính xác suất cao cho các trường hợp tích cực ($y = 1$) và xác suất thấp cho các trường hợp tiêu cực ($y = 0$). Ý tưởng này được thể hiện bằng hàm chi phí được trình bày trong [Phương trình 4-16](#) cho một trường hợp huấn luyện duy nhất x .

Phương trình 4-16. Hàm chi phí của một lần huấn luyện duy nhất

$$\text{log}(\hat{p}) \text{ nếu } y = 1$$
$$c(x) = \begin{cases} -\text{log}(1 - \hat{p}) & \text{nếu } y = 0 \end{cases}$$

Hàm chi phí này hợp lý vì $-\text{log}(t)$ tăng rất lớn khi t tiến đến 0, do đó chi phí sẽ lớn nếu mô hình ước tính xác suất gần bằng 0 cho trường hợp tích cực, và nó cũng sẽ rất lớn nếu mô hình ước tính xác suất gần bằng 1 cho trường hợp tiêu cực. Mặt khác, $-\text{log}(t)$ gần bằng 0 khi t gần bằng 1, do đó chi phí sẽ gần bằng 0 nếu xác suất ước tính gần bằng 0 cho trường hợp tiêu cực hoặc gần bằng 1 cho trường hợp tích cực, đó chính xác là điều chúng ta muốn.

Hàm chi phí trên toàn bộ tập dữ liệu huấn luyện là chi phí trung bình trên tất cả các trường hợp huấn luyện. Nó có thể được viết trong một biểu thức duy nhất được gọi là tổn thất logarit, được thể hiện trong [Phương trình 4-17](#).

Phương trình 4-17. Hàm chi phí hồi quy logistic (logarit của tổn thất)

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m [y(i) \text{log}(\hat{p}(i)) + (1 - y(i)) \text{log}(1 - \hat{p}(i))]$$

CẢNH BÁO

Hàm mất mát logarit không phải được chọn ngẫu nhiên. Có thể chứng minh bằng toán học (sử dụng suy luận Bayes) rằng việc tối thiểu hóa hàm mất mát này sẽ dẫn đến mô hình có xác suất tối ưu cao nhất, giả sử các trường hợp tuân theo phân phối Gaussian xung quanh giá trị trung bình của lớp chúng. Khi sử dụng hàm mất mát logarit, đây là giả định ngầm định mà bạn đang đưa ra. Giả định này càng sai thì mô hình càng bị thiên lệch. Tương tự, khi chúng ta sử dụng MSE để huấn luyện các mô hình hồi quy tuyến tính, chúng ta đã ngầm định rằng dữ liệu hoàn toàn tuyến tính, cộng thêm một số nhiễu Gaussian. Vì vậy, nếu dữ liệu không tuyến tính (ví dụ: bậc hai) hoặc nếu nhiễu không phải là Gaussian (ví dụ: nếu các giá trị ngoại lệ không hiếm theo cấp số mũ), thì mô hình sẽ bị thiên lệch.

Tin xấu là không có phương trình dạng đóng nào được biết đến để tính giá trị của θ sao cho hàm chi phí này được tối thiểu hóa (không có phương trình tương đương với Phương trình Chuẩn). Nhưng tin tốt là hàm chi phí này lồi, vì vậy thuật toán Gradient Descent (hoặc bất kỳ thuật toán tối ưu hóa nào khác) được đảm bảo sẽ tìm thấy điểm cực tiểu toàn cục (nếu tốc độ học không quá lớn và bạn chờ đủ lâu). Các đạo hàm riêng của hàm chi phí đối với tham số mô hình j θ được cho bởi [Phương trình 4-18](#).

$$\frac{\partial J}{\partial \theta_j}$$

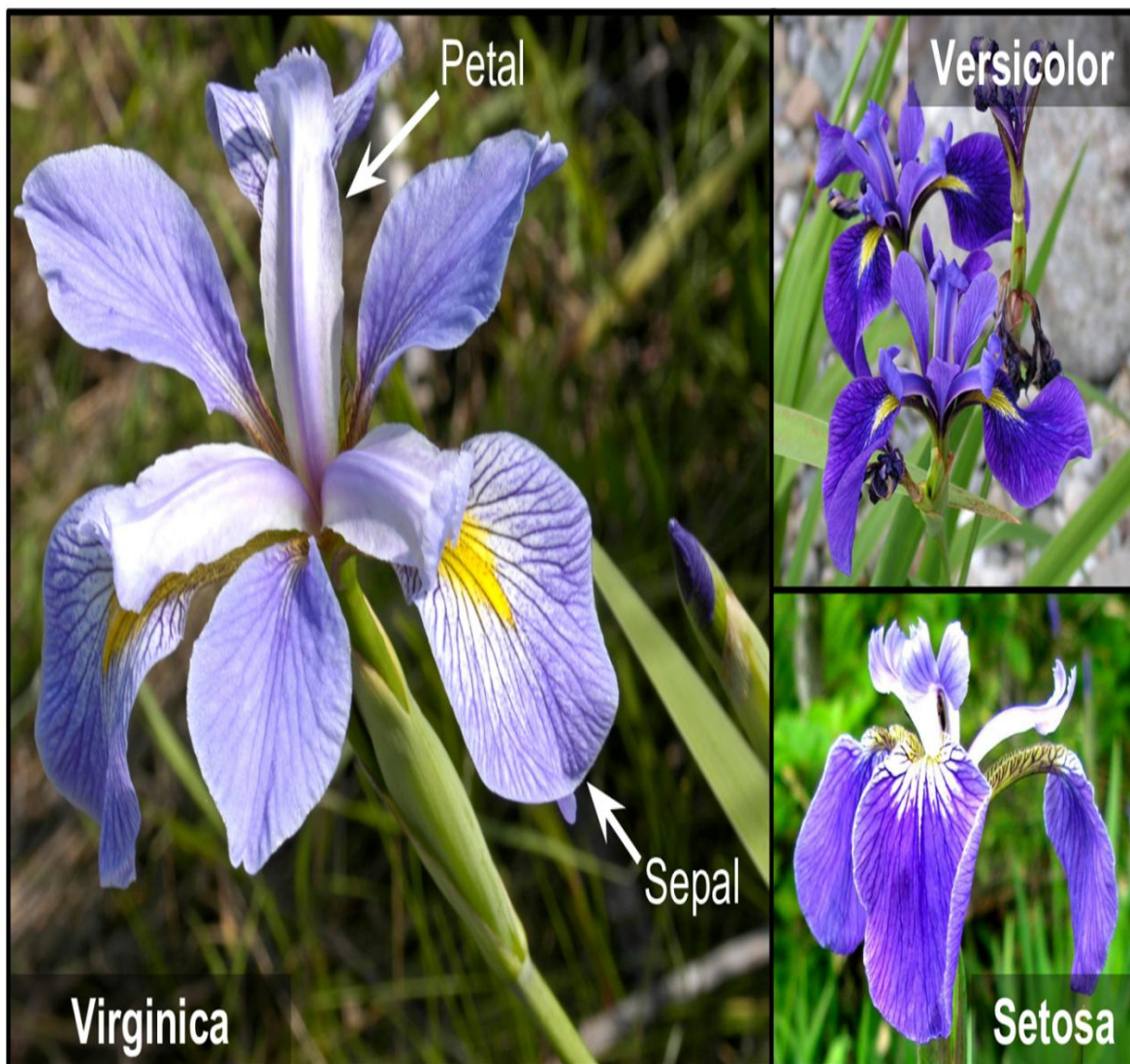
Phương trình 4-18. Đạo hàm riêng của hàm chi phí hậu cần

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\sigma(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Phương trình này trông rất giống **Phương trình 4-5**: với mỗi trường hợp, nó tính toán lỗi dự đoán và nhân với giá trị đặc trưng x_j , sau đó tính trung bình trên tất cả các trường hợp huấn luyện. Khi bạn có vectơ gradient chứa tất cả các đạo hàm riêng, bạn có thể sử dụng nó trong thuật toán Giảm độ dốc theo lô (Batch Gradient Descent). Vậy là xong: giờ bạn đã biết cách huấn luyện mô hình Hồi quy Logistic. Đối với Giảm độ dốc ngẫu nhiên (Stochastic GD), bạn sẽ xử lý từng trường hợp một, còn đối với Giảm độ dốc theo lô nhỏ (Mini-batch GD), bạn sẽ sử dụng từng lô nhỏ một.

Ranh giới quyết định

Chúng ta hãy sử dụng bộ dữ liệu iris để minh họa cho Hồi quy Logistic. Đây là một bộ dữ liệu nổi tiếng chứa chiều dài và chiều rộng của đài hoa và cánh hoa của 150 bông hoa iris thuộc ba loài khác nhau: Iris setosa, Iris versicolor và Iris virginica (xem **Hình 4-22**).



Hình 4-22. Hoa của ba loài cây diên vĩ.

Hãy thử xây dựng một bộ phân loại để phát hiện loại Iris virginica chỉ dựa trên đặc điểm chiều rộng cánh hoa. Trước tiên, hãy...
Tải dữ liệu lên và xem qua nhanh nhé:

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> danh sách(iris)
['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names',
 'filename', 'data_module']
>>> iris.data.head(3)
    Chiều dài đài hoa (cm)  Chiều rộng đài hoa (cm)  Chiều dài cánh hoa (cm)  Chiều rộng cánh hoa (cm)
0  5.1  1.4  0.2  3.5
1  4.9  1.4  0.2  3.0
2  4.7  1.3  0.2  3.2
```

Bây giờ chúng ta hãy chia dữ liệu và huấn luyện mô hình Hồi quy Logistic trên tập dữ liệu huấn luyện:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X = iris.data[["chiều rộng cánh hoa (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

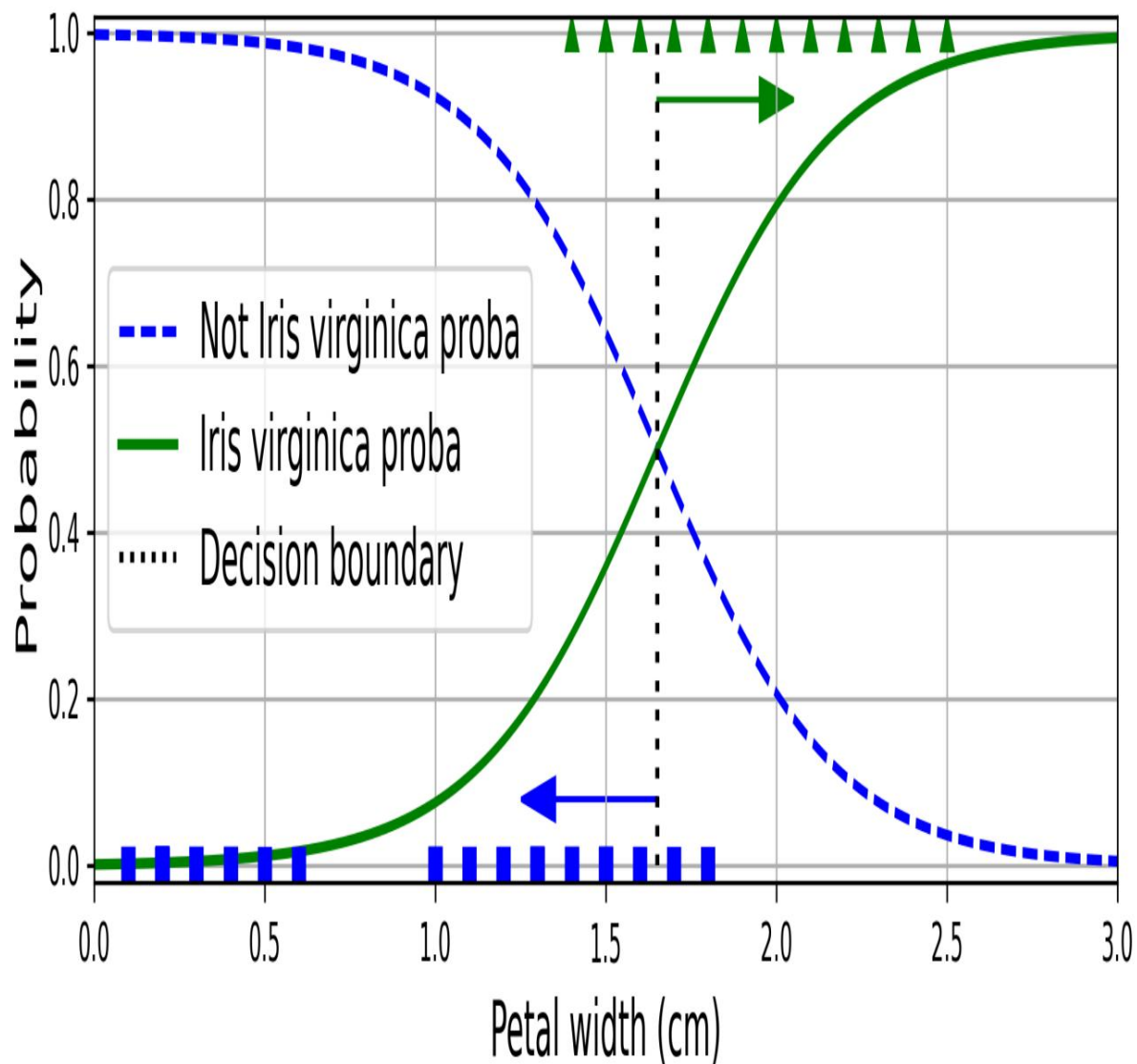
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
```

Hãy cùng xem xét xác suất ước tính của mô hình đối với các bông hoa có chiều rộng cánh hoa dao động từ 0 cm đến 3 cm.
(Hình 4-23): 13

```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1) # reshape để có được một vectơ cột
y_proba = log_reg.predict_proba(X_new)
ranh_giới_quyết_định = X_new[y_proba[:, 1] >= 0.5][0, 0]

plt.plot(X_new, y_proba[:, 0], "b--", linewidth=2,
        nhãn="Không phải Iris virginica proba")
plt.plot(X_new, y_proba[:, 1], "g-", linewidth=2, label="Iris virginica proba")
plt.plot([decision_boundary, decision_boundary], [0, 1], "k:", linewidth=2,
        nhãn="Ranh giới quyết định")

[...] # Làm đẹp hình ảnh: thêm lưới, nhãn, trục, chú thích, mũi tên và mẫu
plt.show()
```

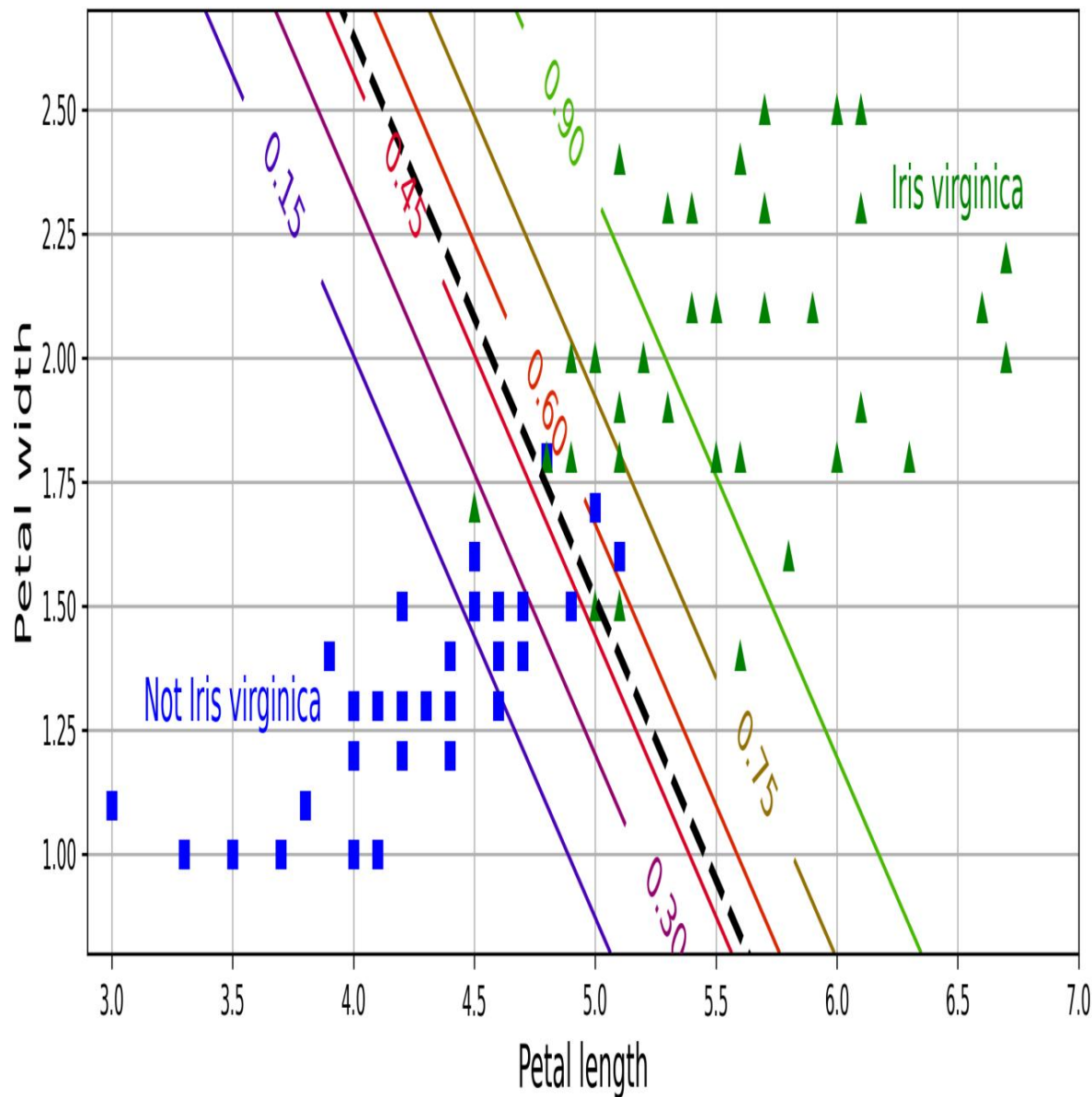



Hình 4-23. Xác suất ước tính và ranh giới quyết định

Chiều rộng cánh hoa của loài *Iris virginica* (được biểu thị bằng hình tam giác) dao động từ 1,4 cm đến 2,5 cm, trong khi các loài hoa diên vĩ khác (được biểu thị bằng hình vuông) thường có chiều rộng cánh hoa nhỏ hơn, dao động từ 0,1 cm đến 1,8 cm. Hãy lưu ý rằng có một chút sự chồng chéo. Ở kích thước trên khoảng 2 cm, bộ phân loại rất tự tin rằng bông hoa là *Iris virginica* (nó đưa ra xác suất cao cho lớp đó), trong khi ở kích thước dưới 1 cm, nó rất tự tin rằng đó không phải là *Iris virginica* (xác suất cao cho lớp "Không phải *Iris virginica*"). Ở giữa hai thái cực này, bộ phân loại không chắc chắn. Tuy nhiên, nếu bạn yêu cầu nó dự đoán lớp (sử dụng phương thức `predict()` thay vì phương thức `predict_proba()`), nó sẽ trả về lớp nào có khả năng xảy ra cao nhất. Do đó, có một ranh giới quyết định ở khoảng 1,6 cm, nơi cả hai xác suất đều bằng 50%: nếu chiều rộng cánh hoa lớn hơn 1,6 cm, bộ phân loại sẽ dự đoán rằng bông hoa là *Iris virginica*, và ngược lại, nó sẽ dự đoán rằng đó không phải là (ngay cả khi nó không hoàn toàn tự tin):

```
>>> decision_boundary
1.6516516516516517 >>>
log_reg.predict([[1.7], [1.5]]) array([ True,
      False])
```


Hình 4-24 hiển thị cùng một tập dữ liệu, nhưng lần này hiển thị hai đặc điểm: chiều rộng và chiều dài cánh hoa. Sau khi được huấn luyện, bộ phân loại Hồi quy Logistic có thể, dựa trên hai đặc điểm này, ước tính xác suất một bông hoa mới là Iris virginica. Đường nét đứt biểu thị các điểm mà mô hình ước tính xác suất 50%: đây là ranh giới quyết định của mô hình. Lưu ý rằng đó là một ranh giới tuyến tính. Mỗi đường thẳng song song biểu thị các điểm mà mô hình đưa ra một xác suất cụ thể, từ 15% (dưới cùng bên trái) đến 90% (trên cùng bên phải). Theo mô hình, tất cả các bông hoa nằm ngoài đường thẳng phía trên bên phải đều có hơn 90% khả năng là Iris virginica.



Hình 4-24. Đường ranh giới quyết định tuyến tính

Cũng giống như các mô hình tuyến tính khác, mô hình hồi quy logistic có thể được điều chỉnh bằng cách sử dụng hình phạt ℓ_1 hoặc ℓ_2 . Trên thực tế, Scikit-Learn thêm hình phạt ℓ_2 theo mặc định.

GHI CHÚ

Siêu tham số điều khiển độ mạnh của quá trình điều chỉnh trong mô hình LogisticRegression của Scikit-Learn không phải là alpha (như trong các mô hình tuyến tính khác), mà là nghịch đảo của nó: C. Giá trị của C càng cao, mô hình càng ít được điều chỉnh.

Hồi quy Softmax: Mô hình hồi

quy logistic có thể được tổng quát hóa để hỗ trợ trực tiếp nhiều lớp, mà không cần phải huấn luyện và kết hợp nhiều bộ phân loại nhị phân (như đã thảo luận trong [Chương 3](#)). Điều này được gọi là hồi quy Softmax, hoặc hồi quy logistic đa thức.

Ý tưởng rất đơn giản: khi được cung cấp một trường hợp x , mô hình Hồi quy Softmax trước tiên tính toán điểm số $s_k(x)$ cho mỗi lớp k , sau đó ước tính xác suất của mỗi lớp bằng cách áp dụng hàm softmax (còn được gọi là hàm mũ chuẩn hóa) cho các điểm số. Phương trình để tính $s_k(x)$ sẽ trông quen thuộc, vì nó giống như phương trình dự đoán Hồi quy tuyến tính (xem [Phương trình 4-19](#)).

Phương trình 4-19. Điểm Softmax cho lớp k

$$s_k(x) = \theta_k^T x$$

Lưu ý rằng mỗi lớp có vectơ tham số θ riêng trong ma trận tham số Θ . ^(k) Tất cả các vectơ này thường được lưu trữ dưới dạng các hàng.

Sau khi tính toán điểm số của mỗi lớp cho trường hợp x , bạn có thể ước tính xác suất p^k rằng trường hợp đó thuộc lớp k bằng cách chạy các điểm số qua hàm softmax ([Phương trình 4-20](#)). Hàm này tính lũy thừa của mỗi điểm số, sau đó chuẩn hóa chúng (chia cho tổng của tất cả các lũy thừa). Các điểm số này thường được gọi là logit hoặc log-odds (mặc dù thực tế chúng là log-odds chưa được chuẩn hóa).

Phương trình 4-20. Hàm Softmax

$$p^k = \sigma(s(x))_k = \frac{\exp(s_k(x))}{\sum_{j=1}^K \exp(s_j(x))}$$

Trong phương trình này:

- K là số lượng lớp học.
- $s(x)$ là một vectơ chứa điểm số của mỗi lớp cho trường hợp x .
- $\sigma(s(x))$ là xác suất ước tính rằng trường hợp x thuộc về lớp k , dựa trên điểm số của mỗi lớp đối với trường hợp k đó.

Giống như bộ phân loại Hồi quy Logistic, theo mặc định, bộ phân loại Hồi quy Softmax dự đoán lớp có xác suất ước tính cao nhất (đơn giản là lớp có điểm số cao nhất), như được thể hiện trong [Phương trình 4-21](#).

Phương trình 4-21. Dự đoán phân loại hồi quy Softmax

$$\hat{y} = \underset{k}{\operatorname{argmax}} \sigma(s(x))_k = \underset{k}{\operatorname{argmax}} s_k(x) = \underset{k}{\operatorname{argmax}} (\theta_k^T x)$$

Toán tử argmax trả về giá trị của một biến làm tối đa hóa một hàm. Trong phương trình này, nó trả về giá trị của k làm tối đa hóa xác suất ước tính $\sigma(s(x))_k$.

MẸO

Bộ phân loại Softmax Regression chỉ dự đoán một lớp tại một thời điểm (tức là nó là đa lớp, không phải đa đầu ra), vì vậy nó chỉ nên được sử dụng với các lớp loại trừ lẫn nhau, chẳng hạn như các loài thực vật khác nhau. Bạn không thể sử dụng nó để nhận dạng nhiều người trong một bức ảnh.

Giờ bạn đã hiểu cách mô hình ước tính xác suất và đưa ra dự đoán, hãy cùng xem xét quá trình huấn luyện. Mục tiêu là xây dựng một mô hình ước tính xác suất cao cho lớp mục tiêu (và do đó, xác suất thấp).